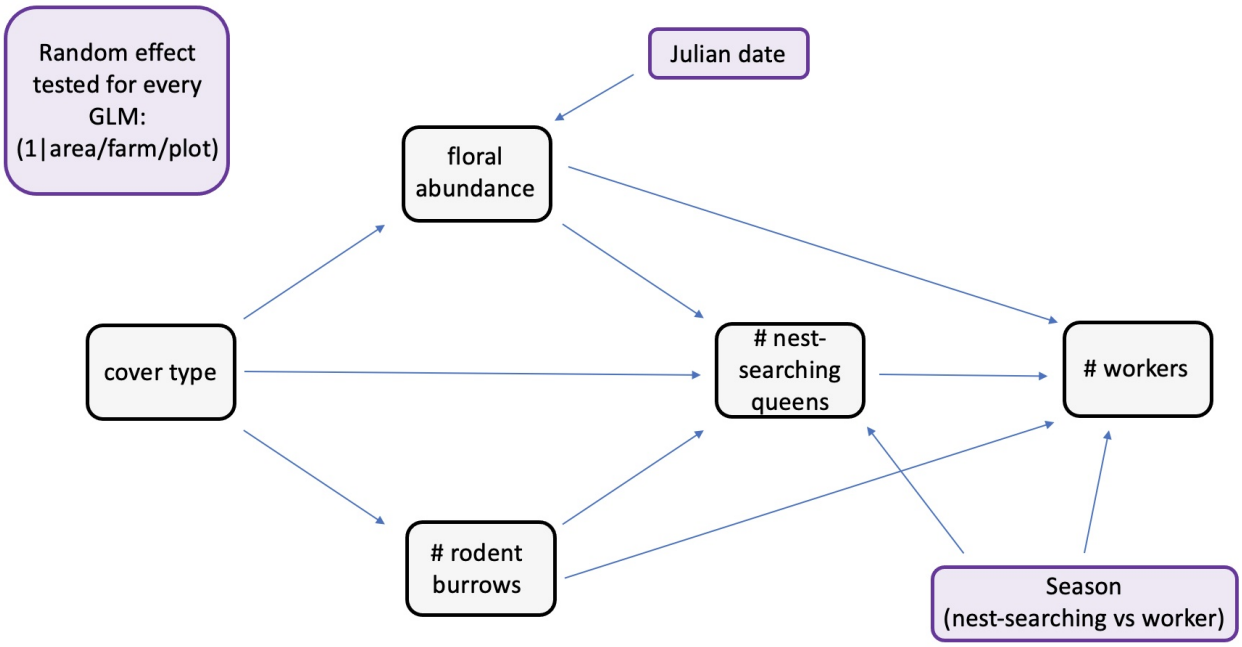
ch1_glm_clean

Predicted path diagram



Load and clean data

```
#load packages
library(tidyverse)    #data manipulation, visualization etc. (dplyr, ggplot2, tidyr)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(DHARMa)      #diagnostic tools for mixed regression models
```

```
## This is DHARMa 0.4.7. For overview type '?DHARMa'. For recent changes, type news(package = 'DHARMa')
```

```
library(ordinal)     #models for ordinal outcomes (e.g., cumulative link models)
```

```
##
## Attaching package: 'ordinal'
##
## The following object is masked from 'package:dplyr':
##
##     slice
```

```
library(MASS)        #polr
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##     select
```

```
library(glmmTMB)     #fits generalized linear mixed models, including zero-inflated and overdispersed m
library(car)         #tools for regression diagnostics and hypothesis tests
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
## The following object is masked from 'package:purrr':
##
##     some
```

```
library(emmeans)     #calculates predicted results from models
```

```
## Welcome to emmeans.
## Caution: You lose important information if you filter this package's results.
## See '? untidy'
```

```
#read in data
data <- read.csv("06_cleandata/combined_plot_data.csv")

#change character columns to factors
surveys <- data %>%
   mutate(
```

```
    site_id = as.factor(site_id),
    round = as.factor(round),
    plot_id = as.factor(plot_id),
    bee_date = as.Date(bee_date),
    bee_season_type = as.factor(bee_season_type),
    area = as.factor(area),
    cover_type = as.factor(cover_type))

#scale continuous predictors
surveys$julian.date_scaled <- as.numeric(scale(surveys$julian.date))
surveys$bare_perc_scaled <- as.numeric(scale(surveys$bare_perc))
surveys$grass_perc_scaled <- as.numeric(scale(surveys$grass_perc))
surveys$forb_perc_scaled <- as.numeric(scale(surveys$forb_perc))
surveys$grass_forb_perc_scaled <- as.numeric(scale(surveys$grass_forb_perc))
surveys$grass_h_scaled <- as.numeric(scale(surveys$grass_h))
surveys$forb_h_scaled <- as.numeric(scale(surveys$forb_h))
surveys$floral_perc_scaled <- as.numeric(scale(surveys$floral_perc))
surveys$grass_forb_floral_perc_scaled <- as.numeric(scale(surveys$grass_forb_floral_perc))
surveys$bombus_floral_perc_scaled <- as.numeric(scale(surveys$bombus_floral_perc))
surveys$flood_perc_scaled <- as.numeric(scale(surveys$flood_perc))
surveys$dry_root_perc_scaled <- as.numeric(scale(surveys$dry_root_perc))
surveys$bryo_perc_scaled <- as.numeric(scale(surveys$bryo_perc))
surveys$bare_flood_dry_bryo_perc_scaled <- as.numeric(scale(surveys$bare_flood_dry_bryo_perc))
surveys$floral_richness_scaled <- as.numeric(scale(surveys$floral_richness))
surveys$bombus_floral_richness_scaled <- as.numeric(scale(surveys$bombus_floral_richness))
surveys$bombus_floral_abun_scaled <- as.numeric(scale(surveys$bombus_floral_abun))
surveys$bombus_floral_abun_nv_scaled <- as.numeric(scale(surveys$bombus_floral_abun_nv))
surveys$nest_searching_queens_scaled <- as.numeric(scale(surveys$nest_searching_queens))
surveys$workers_scaled <- as.numeric(scale(surveys$workers))
surveys$forage_vaco_scaled <- as.numeric(scale(surveys$forage_vaco))
surveys$num_burrows_scaled <- as.numeric(scale(surveys$num_burrows))
surveys$num_active_burrows_scaled <- as.numeric(scale(surveys$num_active_burrows))
surveys$num_inactive_burrows_scaled <- as.numeric(scale(surveys$num_inactive_burrows))
surveys$num_active_dm_scaled <- as.numeric(scale(surveys$num_active_dm))
surveys$num_dm_size_scaled <- as.numeric(scale(surveys$num_dm_size))
surveys$num_inactive_dm_scaled <- as.numeric(scale(surveys$num_inactive_dm))
surveys$prop_inactive_burrows_scaled <- as.numeric(scale(surveys$prop_inactive_burrows))
surveys$num_runway_burrows_scaled <- as.numeric(scale(surveys$num_runway_burrows))

#create variable for proportion of vacant burrows (# inactive / total)
surveys$prop_inactive <- surveys$num_inactive_burrows / (surveys$num_inactive_burrows + surveys$num_acti
#scale
surveys$prop_inactive_scaled <- as.numeric(scale(surveys$prop_inactive))
#check correlation
cor(
  surveys$prop_inactive,
  surveys$num_burrows,
  use = "complete.obs"
)
```

```
## [1] 0.06838928
```

```
##r = 0.06. This means that the vacancy probability and rodent burrow abundance are largely independent

#filter to only include field data
field_data <- surveys %>%
  filter(plot_id != "d")
```

Rodent burrows vs field management (%grass+forb)

Since the number of rodent burrows was count data, I knew I had to use either a poisson or negative binomial GLM family. I started with the simplest family (poisson) first. I didn't include julian.date as a covariate in this model since I assumed that the number of rodent burrows remains relatively consistent over the course of a season (hence why I only sampled each farm once).

Steps

1) Try different random effect structures in the poisson. Compare alternative random effect structures using maximum likelihood (ML) via AIC. I did not include plot as a random effect structure since there was only one observation per plot for rodent burrow data.

- best fit = 1|site_id –> **area not included**

```
burrows_full <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
                          (1|area/site_id),
                        family = poisson,
                        data = field_data)

burrows_siteOnly <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
                          (1|site_id),
                        family = poisson,
                        data = field_data)

burrows_areaOnly <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
                          (1|area),
                        family = poisson,
                        data = field_data)


#compare models using maximum likelihood test
anova(burrows_full, burrows_siteOnly, burrows_areaOnly)
```

```
## Data: field_data
## Models:
## burrows_siteOnly: num_burrows ~ grass_forb_perc_scaled + (1 | site_id), zi=~0, disp=~1
## burrows_areaOnly: num_burrows ~ grass_forb_perc_scaled + (1 | area), zi=~0, disp=~1
## burrows_full: num_burrows ~ grass_forb_perc_scaled + (1 | area/site_id), zi=~0, disp=~1
##                  Df    AIC    BIC   logLik deviance  Chisq Chi Df Pr(>Chisq)
## burrows_siteOnly  3 1670.5 1679.3  -832.25   1664.5
## burrows_areaOnly  3 3306.4 3315.2 -1650.18   3300.4    0.0      0          1
## burrows_full      4 1672.1 1683.9  -832.05   1664.1 1636.3      1     <2e-16
##
```

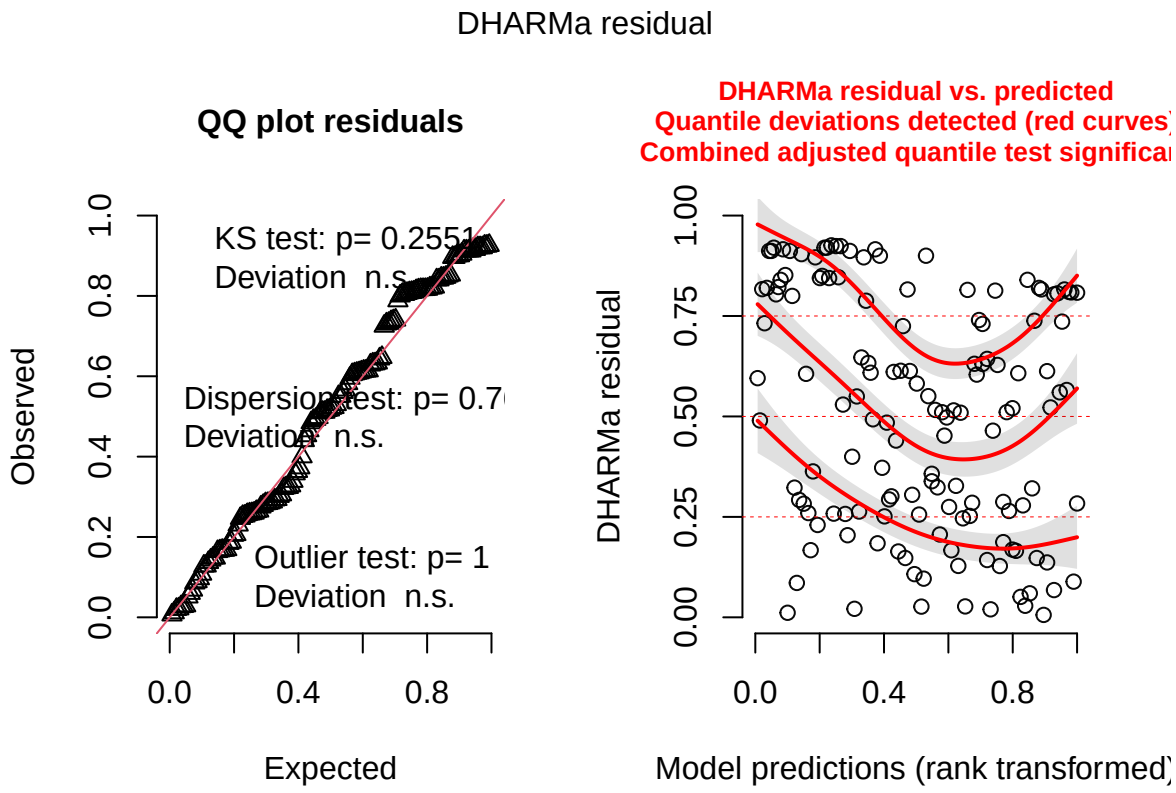
```
## burrows_siteOnly
## burrows_areaOnly
## burrows_full     ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##results tells us the burrows_siteOnly is best model (lowest AIC)
```

2) Check diagnostics of the model using DHARMa package.

- Combined adjusted quantile test significant

```
#simulate residuals
residuals_burrows_siteOnly <- simulateResiduals(fittedModel = burrows_siteOnly, plot = TRUE)
```

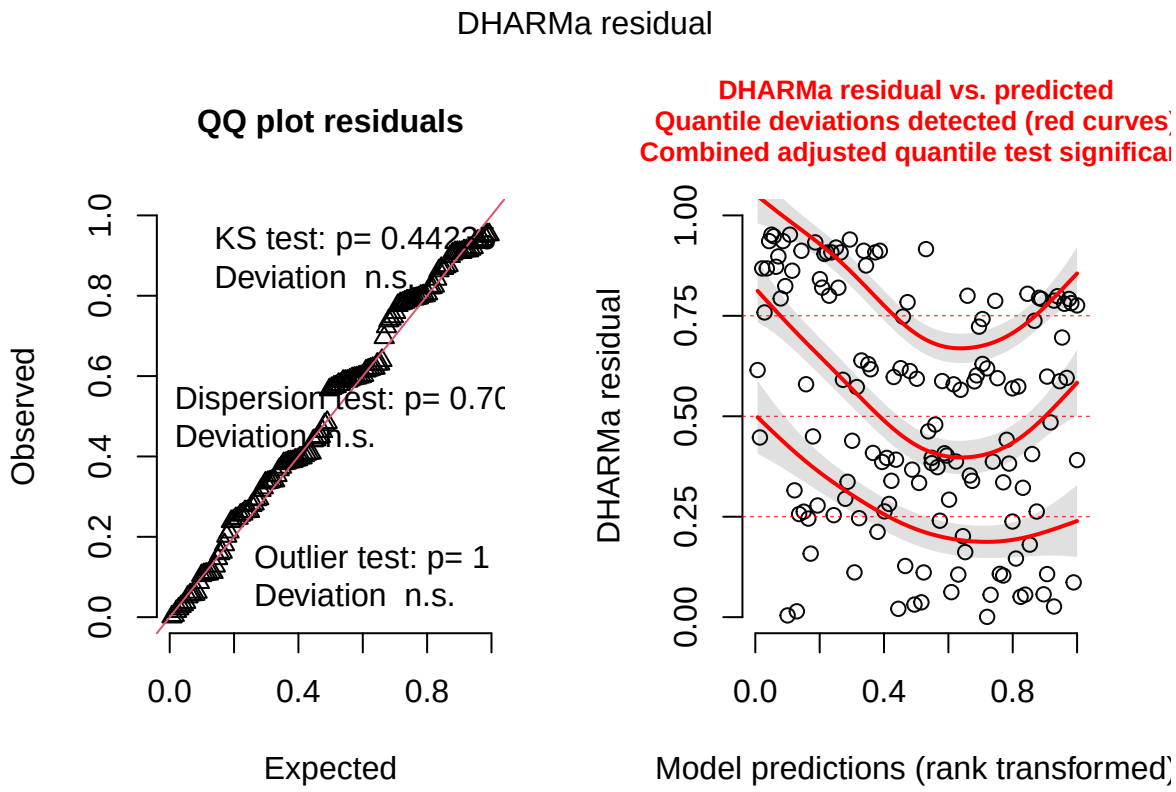


3) Try negative binomial instead and check diagnostics.

- Combined adjusted quantile test still significant.

```
burrows_siteOnly_negBin <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
                             (1|site_id),
                           family = nbinom2,
                           data = field_data)
```

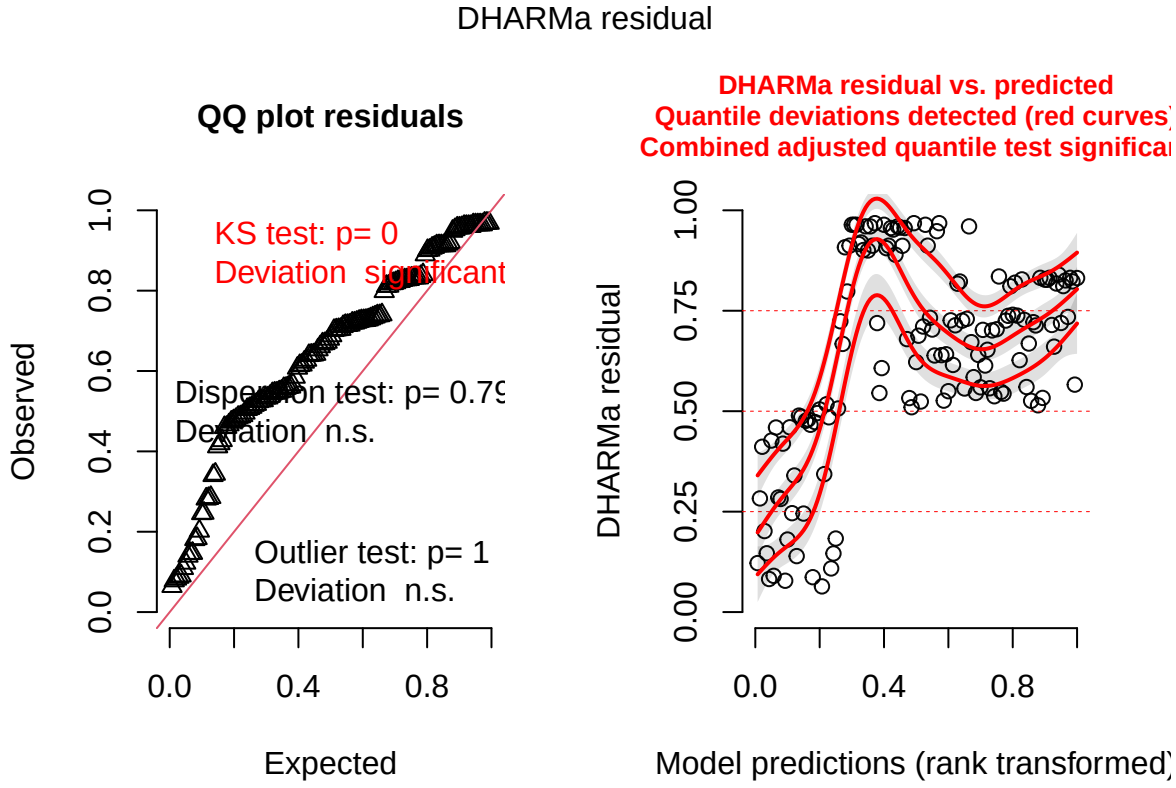
```
#simulate residuals
residuals_burrows_siteOnly_negBin <- simulateResiduals(fittedModel = burrows_siteOnly_negBin, plot = TRU
```



4) Try zero-inflated poisson and check diagnostics.

- combined adjusted quantile test still significant.

```
burrows_zi_pois <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
                             (1|site_id),
                           ziformula = ~ grass_forb_perc_scaled +
                             (1|site_id),
                           family = poisson,
                           data = field_data)

#simulate residuals
residuals_burrows_zi <- simulateResiduals(fittedModel = burrows_zi_pois, plot = TRUE)
```

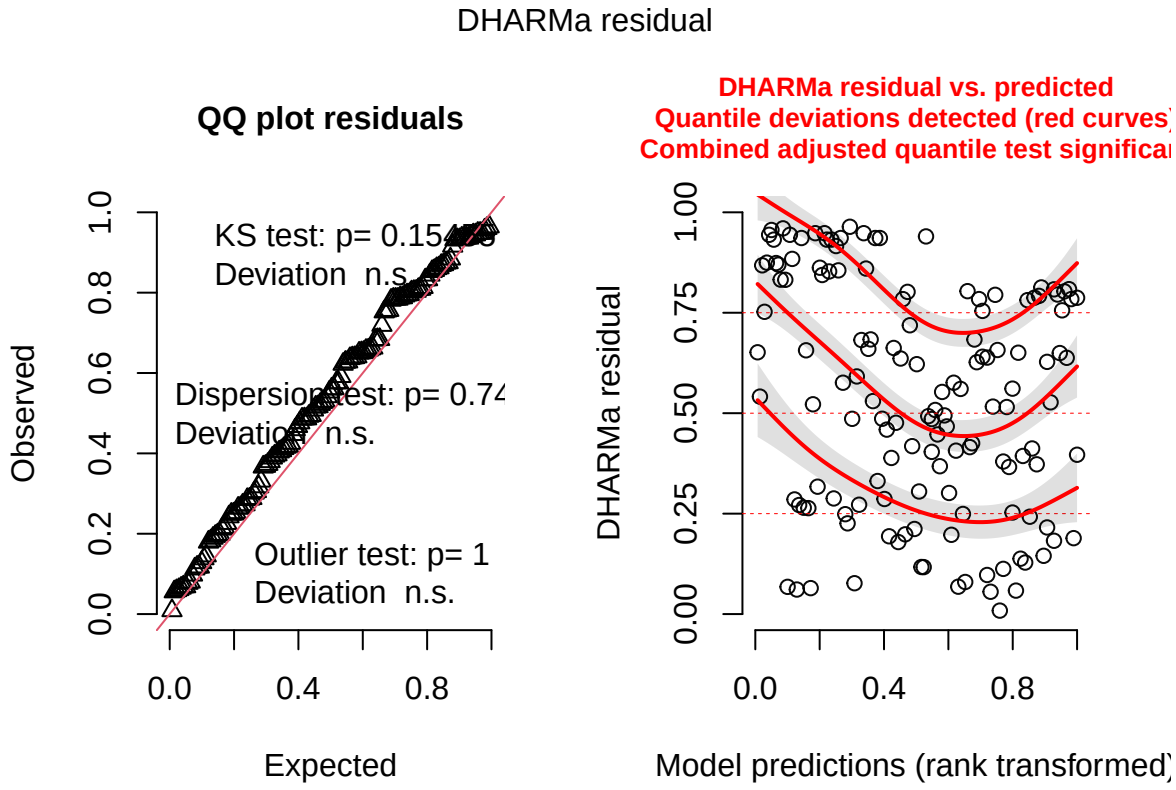


5) Try zero-inflated negative binomial and check diagnostics.

- combined adjusted quantile test still significant.

```
burrows_zi_nbin <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
                             (1|site_id),
                           ziformula = ~ grass_forb_perc_scaled +
                             (1|site_id),
                           family = nbinom2,
                           data = field_data)

#simulate residuals
residuals_burrows_zi_nbin <- simulateResiduals(fittedModel = burrows_zi_nbin, plot = TRUE)
```

```
## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully
```




```
#signficant KS test and combined adjusted quantile test
```

6) Stick with poisson but try using categorical cover_type as predictor instead of % grass/forb cover and try different random effect structures.

- models using continuous percent grass/forb cover as a predictor showed poor fit and significant residual patterns, likely due to the small sample size (n = 24). Because sites were grouped into two distinct habitat types—grassy plots with high vegetation cover and bare plots with little or no cover—the variable was reclassified as a categorical factor ("grass" vs. "bare"). This simplified model structure improved residual diagnostics and provided a clearer ecological interpretation of differences in burrow abundance between habitat types.

```
burrows_cover_full <- glmmTMB(num_burrows ~ cover_type +
                              (1|area/site_id),
                            family = poisson,
                            data = field_data)

burrows_cover_siteOnly <- glmmTMB(num_burrows ~ cover_type +
                              (1|site_id),
                            family = poisson,
                            data = field_data)

burrows_cover_areaOnly <- glmmTMB(num_burrows ~ cover_type +
                              (1|area),
                            family = poisson,
```

```
                          data = field_data)


#compare models using maximum likelihood test
anova(burrows_cover_full, burrows_cover_siteOnly, burrows_cover_areaOnly)
```

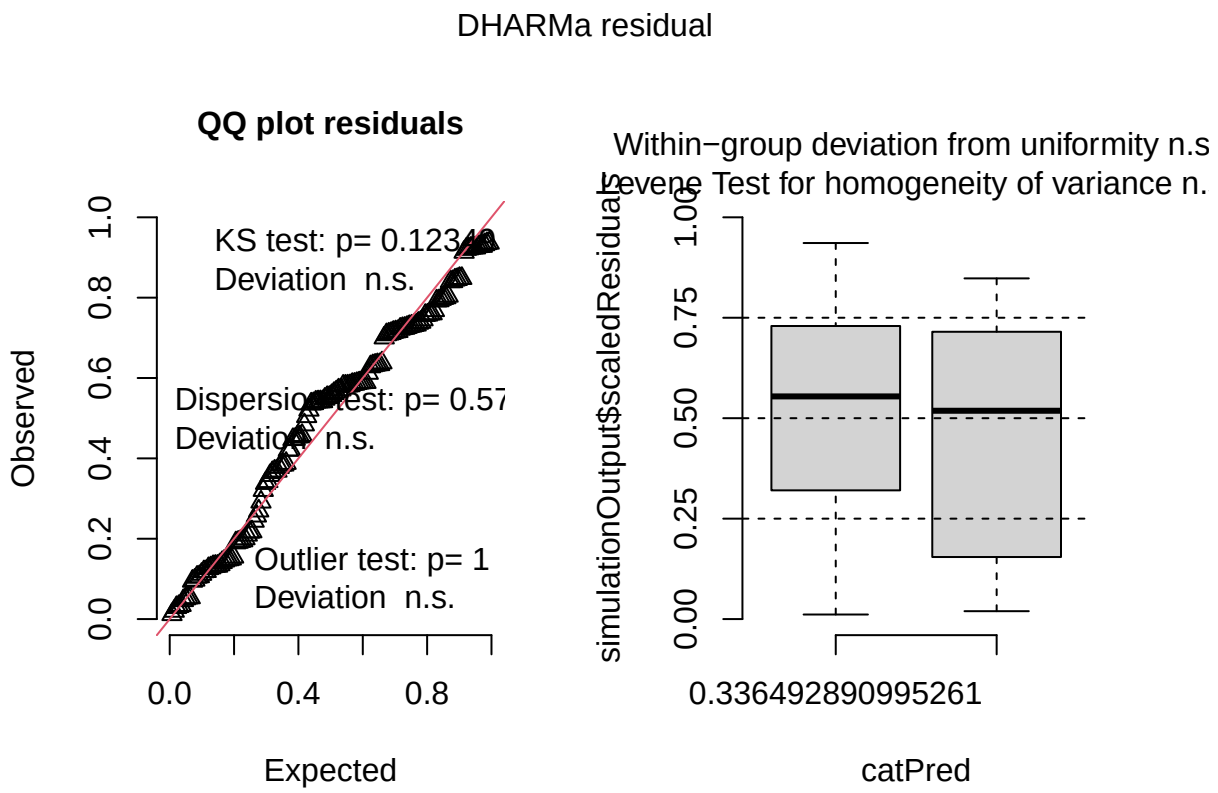
```
## Data: field_data
## Models:
## burrows_cover_siteOnly: num_burrows ~ cover_type + (1 | site_id), zi=~0, disp=~1
## burrows_cover_areaOnly: num_burrows ~ cover_type + (1 | area), zi=~0, disp=~1
## burrows_cover_full: num_burrows ~ cover_type + (1 | area/site_id), zi=~0, disp=~1
##                         Df    AIC    BIC   logLik deviance  Chisq Chi Df
## burrows_cover_siteOnly   3 1668.2 1677.0  -831.11   1662.2
## burrows_cover_areaOnly   3 2664.6 2673.4 -1329.28   2658.6   0.00      0
## burrows_cover_full       4 1667.7 1679.5  -829.87   1659.7 998.81      1
##                         Pr(>Chisq)
## burrows_cover_siteOnly
## burrows_cover_areaOnly          1
## burrows_cover_full         <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##full model is best (lowest AIC)
```

7. Check diagnostics

- significant within-group deviations from uniformity

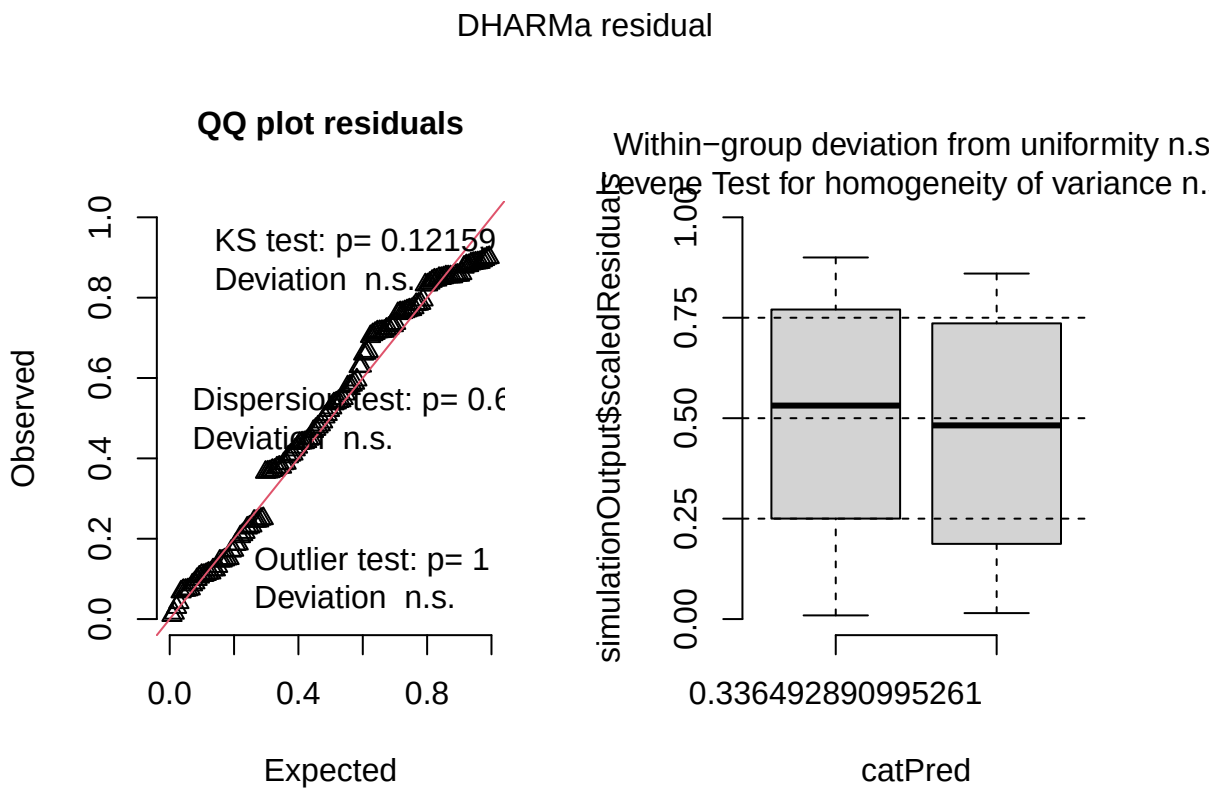
```
#simulate residuals
residuals_burrows_cover_full <- simulateResiduals(fittedModel = burrows_cover_full, plot = TRUE)
```



8. Try simpler random effect structure (site-only) and re-check diagnostics

- No significant problems detected.

```
#simulate residuals
residuals_burrows_cover_siteOnly <- simulateResiduals(fittedModel = burrows_cover_siteOnly, plot = TRUE)
```



9. Plot results

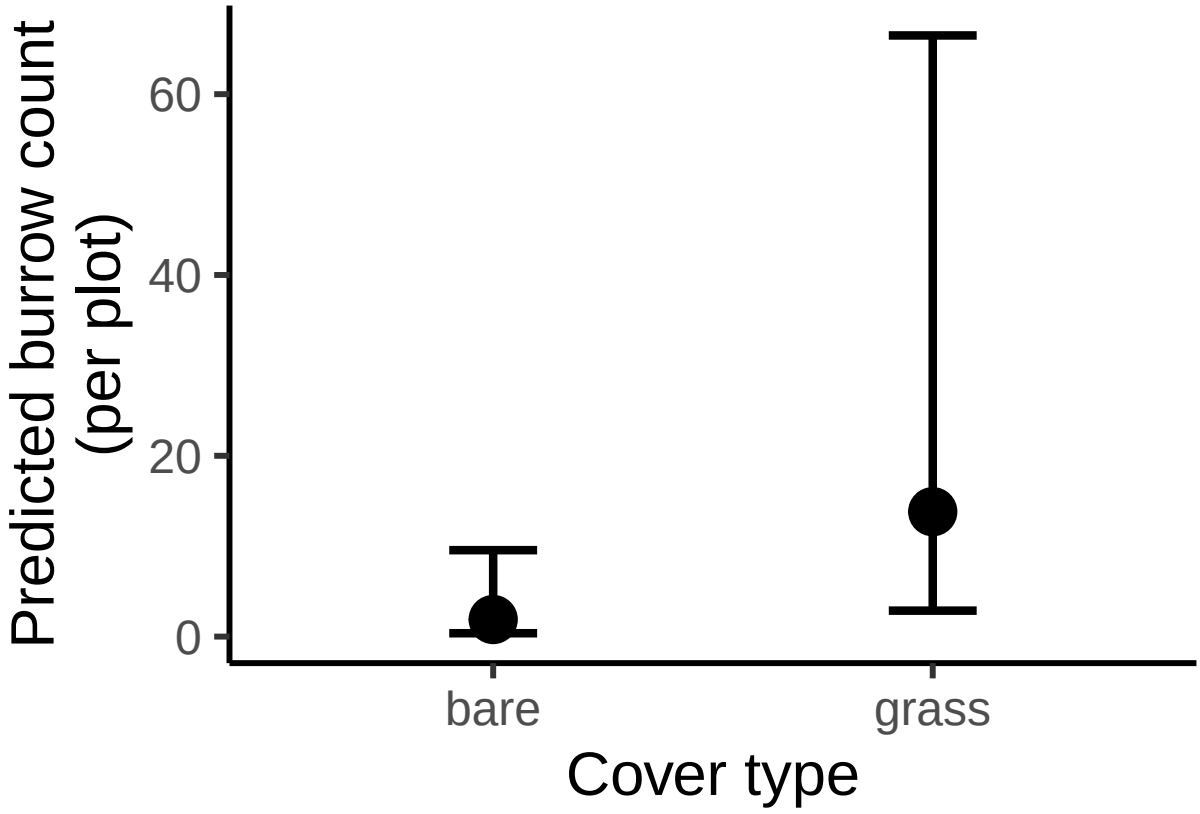
```
summary(burrows_cover_siteOnly)
```

```
##  Family: poisson  ( log )
## Formula:          num_burrows ~ cover_type + (1 | site_id)
## Data: field_data
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    1668.2    1677.0    -831.1    1662.2       137
##
## Random effects:
##
## Conditional model:
##  Groups  Name        Variance Std.Dev.
##  site_id (Intercept) 3.825    1.956
## Number of obs: 140, groups:  site_id, 12
##
## Conditional model:
##                 Estimate Std. Error z value Pr(>|z|)
## (Intercept)       0.6365     0.8271   0.770   0.4415
## cover_typegrass   1.9894     1.1510   1.728   0.0839 .
## ---
## Signif. codes:  0 ’***’ 0.001 ’**’ 0.01 ’*’ 0.05 ’.’ 0.1 ’ ’ 1
```

Plots

Number of rodent burrows vs cover type

```
#create new data grid
rodent_data_cover<- expand.grid(
  cover_type = unique(field_data$cover_type)
)

# Get predicted probabilities and standard errors
pred_rodent_cover <- predict(burrows_cover_siteOnly,
                             newdata = rodent_data_cover,
                             type = "link",
                             se.fit = TRUE,
                             re.form = NA)

# Combine predictions with data frame
pred_rodent_cover_df <- cbind(rodent_data_cover,
                  fit_log = pred_rodent_cover$fit,
                  se_log = pred_rodent_cover$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_rodent_cover_df <- pred_rodent_cover_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )


ggplot(pred_rodent_cover_df, aes(x = cover_type, y = fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Cover type",
       y = "Predicted burrow count\n(per plot)") +
  theme_classic(base_size = 23)
```



Findings

Neither grass + forb % nor julian date had significant effects on rodent burrow abundance. However, cover type showed a marginally insignificant trend (p=0.0839), where grassy sites had a higher abundance of burrows.

Rodent burrow vacancy analysis

I want to determine what influences the proportion of burrows that are inactive (available as nest sites for bumble bees) across my sites.

Steps

1) Try different random effect structures in a binomial model. Compare alternative random effect structures using maximum likelihood (ML) via AIC. I did not include plot as a random effect structure since there was only one observation per plot for rodent burrow data.

```
burrows_vacancy_full <- glmmTMB(
  cbind(num_inactive_burrows, num_active_burrows) #tells the model that successes = number of inac
  ~ cover_type + (1|area/site_id),
  family = binomial(link = "logit"),
  data = field_data)
```

```r
burrows_vacancy_siteOnly <- glmmTMB(
  cbind(num_inactive_burrows, num_active_burrows) #tells the model that successes = number of inact
  ~ cover_type + (1|site_id),
  family = binomial(link = "logit"),
  data = field_data)

burrows_vacancy_areaOnly <- glmmTMB(
  cbind(num_inactive_burrows, num_active_burrows) #tells the model that successes = number of inact
  ~ cover_type + (1|area),
  family = binomial(link = "logit"),
  data = field_data)


#compare models using maximum likelihood test
anova(burrows_vacancy_full, burrows_vacancy_siteOnly, burrows_vacancy_areaOnly)
```

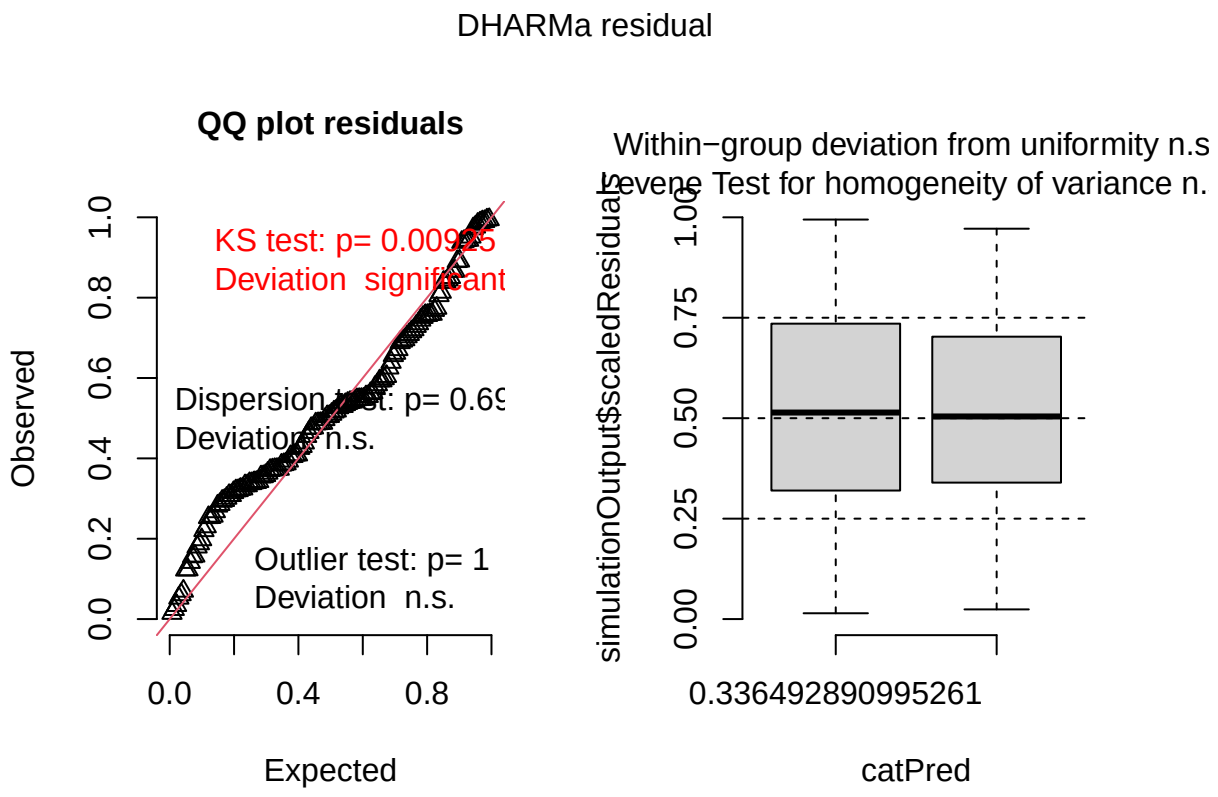
```
## Data: field_data
## Models:
## burrows_vacancy_siteOnly: cbind(num_inactive_burrows, num_active_burrows) ~ cover_type + , zi=~0
## burrows_vacancy_siteOnly:     (1 | site_id), zi=~0, disp=~1
## burrows_vacancy_areaOnly: cbind(num_inactive_burrows, num_active_burrows) ~ cover_type + , zi=~0
## burrows_vacancy_areaOnly:     (1 | area), zi=~0, disp=~1
## burrows_vacancy_full: cbind(num_inactive_burrows, num_active_burrows) ~ cover_type + , zi=~0, di
## burrows_vacancy_full:     (1 | area/site_id), zi=~0, disp=~1
##                          Df    AIC    BIC  logLik deviance  Chisq Chi Df
## burrows_vacancy_siteOnly  3 463.22 472.04 -228.61   457.22
## burrows_vacancy_areaOnly  3 497.71 506.54 -245.86   491.71  0.000      0
## burrows_vacancy_full      4 465.22 476.98 -228.61   457.22 34.495      1
##                          Pr(>Chisq)
## burrows_vacancy_siteOnly
## burrows_vacancy_areaOnly          1
## burrows_vacancy_full      4.273e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
##results tells us the burrows_siteOnly is best model (lowest AIC)
```

2) Check diagnostics

- The diagnostic plot indicates a significant KS test. The significant KS test likely reflects low variability in vacancy probabilities across farms, with vacancy rates clustering near 0.5, rather than substantive lack of fit. No evidence of overdispersion, zero inflation, outliers, or group-level structure was detected.

```r
#simulate residuals
residuals_burrows_vacancy_siteOnly <- simulateResiduals(fittedModel = burrows_vacancy_siteOnly, plot = T
```




```
##KS test significant
```

3. Print results

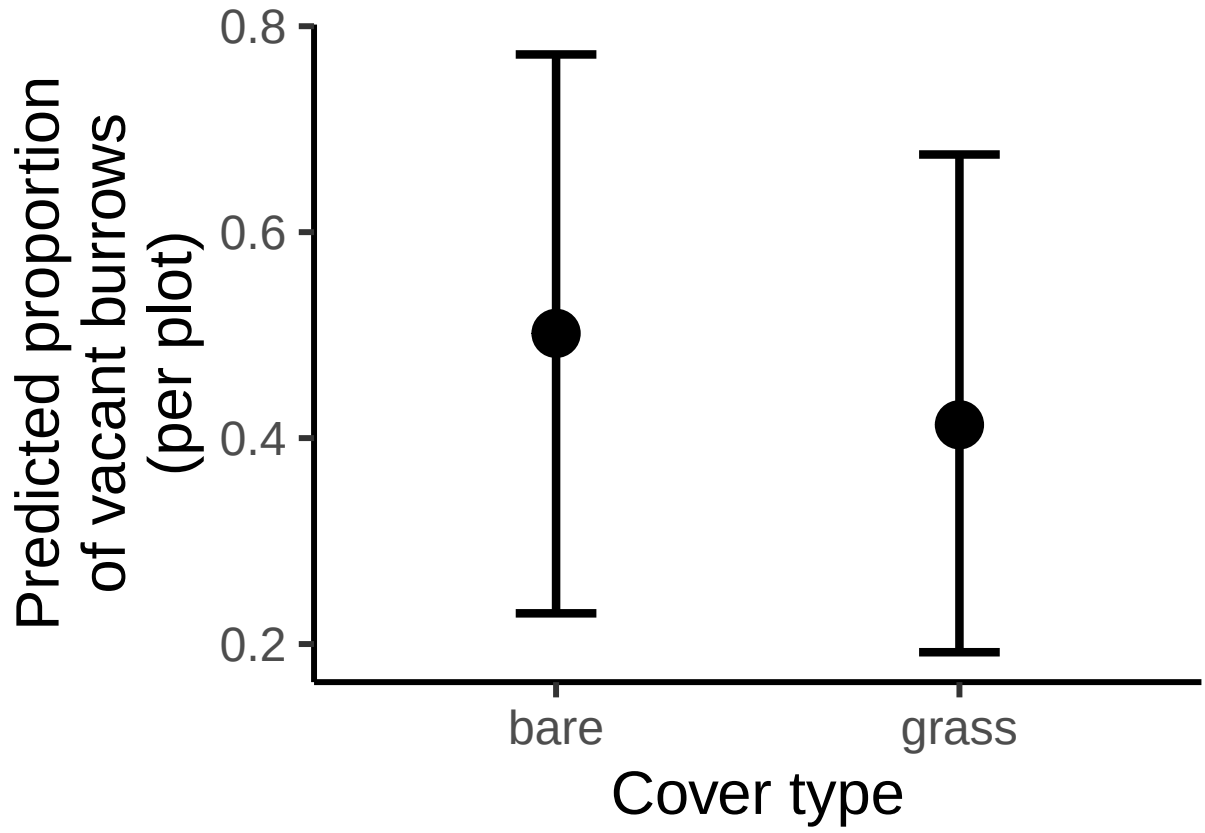
```
summary(burrows_vacancy_siteOnly)
```

```
##  Family: binomial  ( logit )
## Formula:
## cbind(num_inactive_burrows, num_active_burrows) ~ cover_type +
##     (1 | site_id)
## Data: field_data
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     463.2     472.0    -228.6     457.2       137
##
## Random effects:
##
## Conditional model:
##  Groups  Name        Variance Std.Dev.
##  site_id (Intercept) 1.587    1.26
## Number of obs: 140, groups:  site_id, 12
##
## Conditional model:
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)      0.006844   0.620445   0.011    0.991
## cover_typegrass -0.358816   0.831890  -0.431    0.666
```

Plots

Proportion of vacant burrows vs cover type

```
#create new data grid
vacancy_data_cover<- expand.grid(
  cover_type = unique(field_data$cover_type)
)

# Get predicted probabilities and standard errors
pred_vacancy_cover <- predict(burrows_vacancy_siteOnly,
                              newdata = vacancy_data_cover,
                              type = "link",
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_vacancy_cover_df <- cbind(vacancy_data_cover,
                 fit_logit = pred_vacancy_cover$fit,
                 se_logit = pred_vacancy_cover$se.fit)

#calculate predictions and confidence intervals on response scale (not logit scale)
pred_vacancy_cover_df <- pred_vacancy_cover_df %>%
  mutate(
    fit = plogis(fit_logit),
    lower = plogis(fit_logit - 1.96 * se_logit),
    upper = plogis(fit_logit + 1.96 * se_logit)
  )


ggplot(pred_vacancy_cover_df, aes(x = cover_type, y = fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Cover type",
       y = "Predicted proportion\nof vacant burrows\n(per plot)") +
  theme_classic(base_size = 23)
```



Findings

There was not a significant difference in burrow vacancy probability between grassy and bare sites (log-odds difference = –0.36, SE = 0.83, $p = 0.67$). Burrow vacancy probability was approximately 50% across sites, meaning that roughly one in two burrows at blueberry farms in this study were available as potential nest sites for bumble bees.

Floral abundance vs field management (%grass+forb)

Use cumulative link model aka ordinal logistic regression because bombus-relevant floral abundance has:

- discrete levels (0-5 scale) representing binned counts of blooms
- ordered categories (higher numbers represent more blooms)
- unequal intervals between categories

Steps

1) Tried running CLM with different random effect structures (full = 1|area/site/plot) then compared AIC values to determine best model.

- determined that model without random effect had best fit

```
floral_full <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                        julian.date_scaled +
                        (1|area/site_id/plot_id),
                      link = "logit",
                      data = field_data)

floral_noArea <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                        julian.date_scaled +
                        (1|site_id/plot_id),
                      link = "logit",
                      data = field_data)

#floral_plotOnly <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
 #                         julian.date_scaled +
  #                        (1|plot_id),
   #                     link = "logit",
    #                    data = field_data)
##can't use since there are only 2 levels

floral_siteOnly <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                        julian.date_scaled +
                        (1|site_id),
                      link = "logit",
                      data = field_data)

floral_AreaPlot <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                        julian.date_scaled +
                        (1|area/plot_id),
                      link = "logit",
                      data = field_data)
floral_noRE <- polr(as.factor(bombus_floral_abun) ~ cover_type +
                        julian.date_scaled,
                      method = "logistic",
                      data = field_data)

#compare models AIC
AIC(floral_full, floral_noArea, floral_siteOnly, floral_AreaPlot, floral_noRE)
```

```
##                 df      AIC
## floral_full      9 441.1731
## floral_noArea    8 439.1731
## floral_siteOnly  7 437.1731
## floral_AreaPlot  8 439.1731
## floral_noRE      6 435.1731
```

```
##shows that model without random effect has best fit
```

2. Run final model and look at output

```
summary(floral_noRE)
```

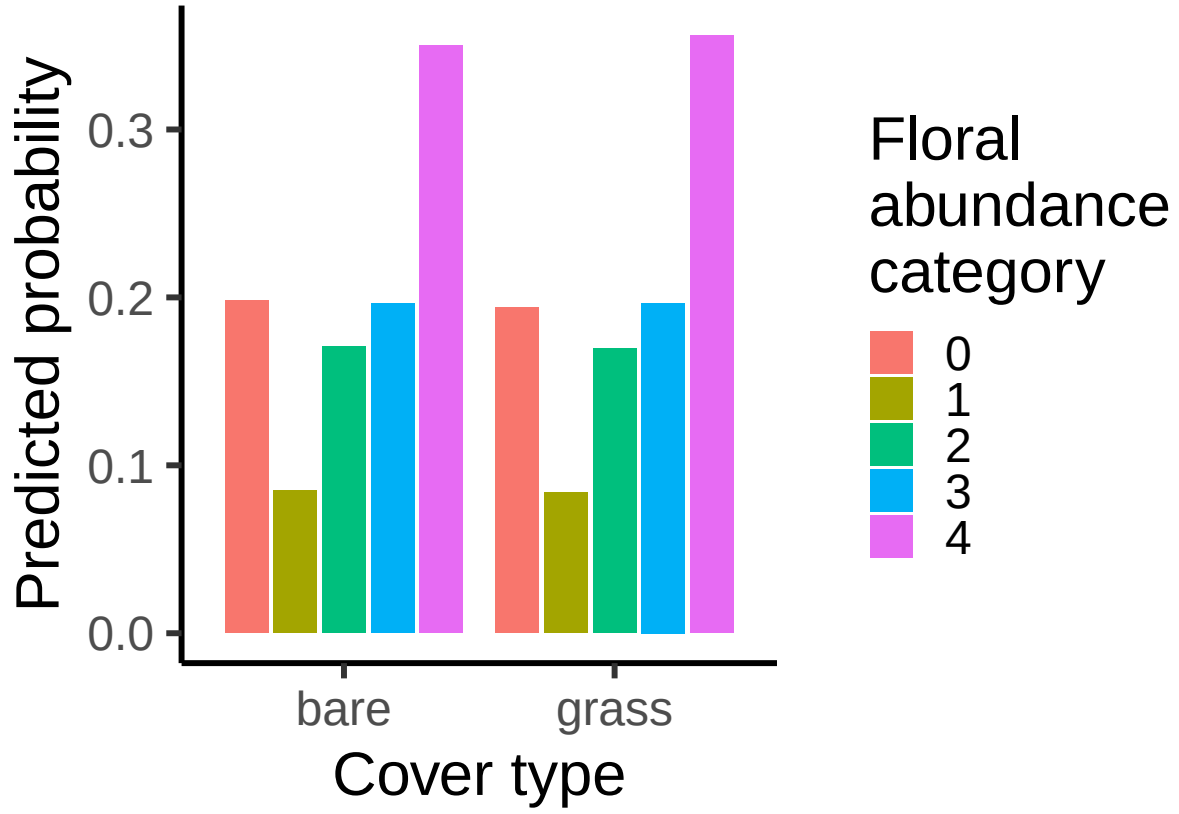
```
##
## Re-fitting to get Hessian
```

```
## Call:
## polr(formula = as.factor(bombus_floral_abun) ~ cover_type + julian.date_scaled,
##     data = field_data, method = "logistic")
##
## Coefficients:
##                       Value Std. Error  t value
## cover_typegrass     0.02647     0.3041  0.08702
## julian.date_scaled -0.23119     0.1527 -1.51417
##
## Intercepts:
##     Value   Std. Error t value
## 0|1 -1.3986  0.2610    -5.3585
## 1|2 -0.9291  0.2414    -3.8493
## 2|3 -0.1853  0.2272    -0.8153
## 3|4  0.6192  0.2332     2.6559
##
## Residual Deviance: 423.1731
## AIC: 435.1731
```

Plots

Vegetation

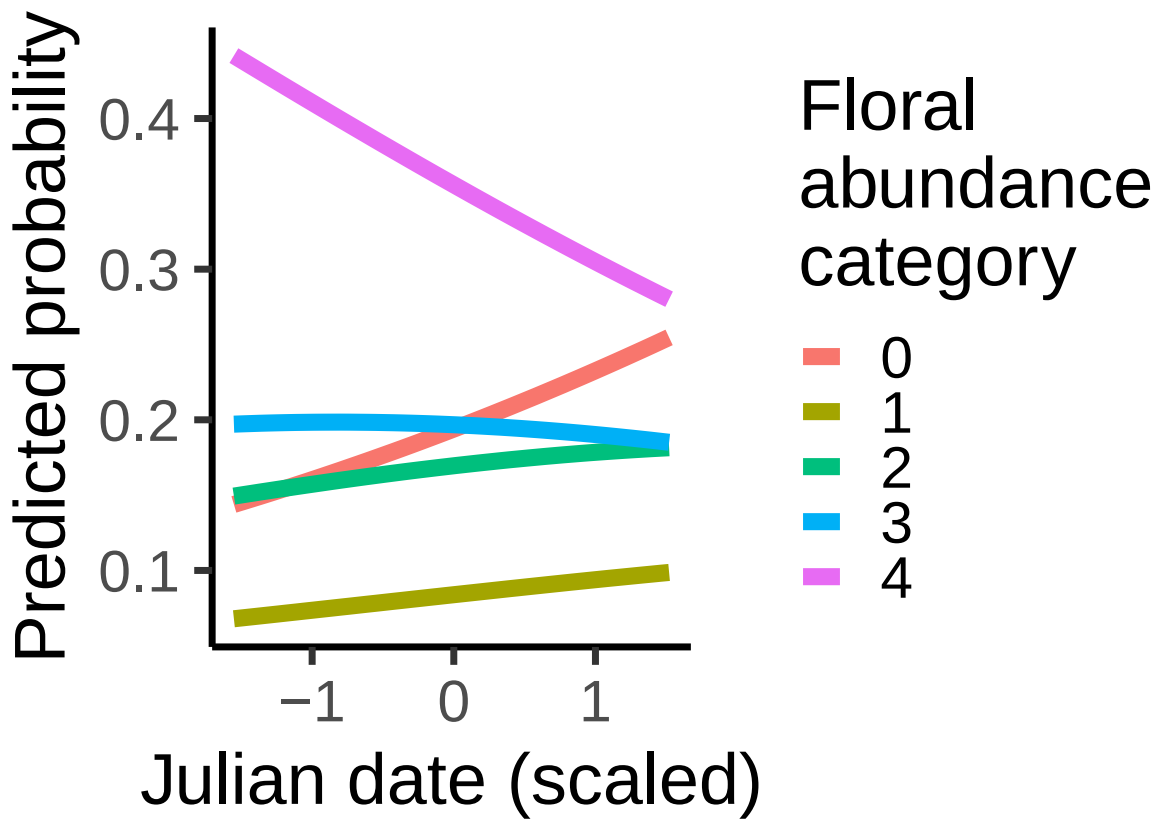
```
floral_veg_data <- data.frame(
  cover_type = factor(c("bare", "grass"), levels = levels(field_data$cover_type)),
  julian.date_scaled = mean(field_data$julian.date_scaled, na.rm = TRUE)
)

# Get predicted response for cover_type, holding other variables at their means
pred_floral_veg <- predict(floral_noRE,
                      newdata = floral_veg_data,
                      type = "prob")


# Convert to long dataframe for plotting

pred_floral_veg_df <- as.data.frame(pred_floral_veg) %>%
  mutate(cover_type = rownames(pred_floral_veg)) %>%
  pivot_longer(cols = -cover_type, names_to = "response", values_to = "prob")

# Convert cover_type to factor with proper labels
pred_floral_veg_df$cover_type <- factor(pred_floral_veg_df$cover_type,
                                   levels = c(1, 2),
                                   labels = c("bare", "grass"))

ggplot(pred_floral_veg_df, aes(x = cover_type, y = prob, fill = response)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.9), width = 0.8) +
  labs(
    x = "Cover type",
    y = "Predicted probability",
    fill = "Floral\nabundance\ncategory"
  ) +
  theme_classic(base_size = 23)
```



Julian date

```
# Define a sequence of values for grass_forb_perc_scaled across its observed range
date_seq <- seq(
  from = min(field_data$julian.date_scaled, na.rm = TRUE),
  to = max(field_data$julian.date_scaled, na.rm = TRUE),
  length.out = 50
)

floral_date_data <- data.frame(
  cover_type = "grass",
  julian.date_scaled = date_seq
)

# Get predicted response for cover_type, holding other variables at their means
pred_floral_date <- predict(floral_noRE,
                      newdata = floral_date_data,
                      type = "prob")


# Convert to long dataframe for plotting
pred_floral_date_df <- as.data.frame(pred_floral_date) %>%
  mutate(julian.date_scaled = floral_date_data$julian.date_scaled) %>%
  pivot_longer(
    cols = 0:5,
    names_to = "floral_abun_cat",
```

```
    values_to = "prob"
  )

ggplot(pred_floral_date_df, aes(x = julian.date_scaled, y = prob, color = floral_abun_cat)) +
  geom_line(linewidth = 3) +
  labs(
    x = "Julian date (scaled)",
    y = "Predicted probability",
    color = "Floral\nabundance\ncategory"
  ) +
  theme_classic(base_size = 27)
```



Findings

No significant effect of either grass+ forb% or julian date.

Number of nest-searching queens vs flowers, rodents, and field management technique

Start by plotting a zero-inflated poisson, since # of nest-searching queens is count data with a lot of zeros. I added bee_season_type as a covariate, since during my exploratory data analyses I saw that there was only a significant different in nest-searching queens among grassy vs bare sites during nesting season.

Steps

1. Try multiple random effects structures

- best fit = no random effect

```
NSQ_full <- glmmTMB(nest_searching_queens ~ cover_type +
                      bombus_floral_abun_scaled +
                      num_burrows_scaled +
                      prop_inactive_scaled +
                      bee_season_type +
                      (1|area/site_id/plot_id),
                    ziformula = ~ cover_type +
                      bombus_floral_abun_scaled +
                      num_burrows_scaled +
                      prop_inactive_scaled +
                      bee_season_type +
                      (1|area/site_id/plot_id),
                    family = poisson,
                    data = field_data)
```

```
## Warning in finalizeTMB(TMBStruc, obj, fit, h, data.tmb.old): Model convergence
## problem; non-positive-definite Hessian matrix. See vignette('troubleshooting')
```

```
NSQ_noArea <- glmmTMB(nest_searching_queens ~ cover_type +
                        bombus_floral_abun_scaled +
                        num_burrows_scaled +
                        prop_inactive_scaled +
                        bee_season_type +
                        (1|site_id/plot_id),
                      ziformula = ~ cover_type +
                        bombus_floral_abun_scaled +
                        num_burrows_scaled +
                        prop_inactive_scaled +
                        bee_season_type +
                        (1|site_id/plot_id),
                      family = poisson,
                      data = field_data)
```

```
## Warning in finalizeTMB(TMBStruc, obj, fit, h, data.tmb.old): Model convergence
## problem; non-positive-definite Hessian matrix. See vignette('troubleshooting')
```

```
NSQ_plotOnly <-  glmmTMB(nest_searching_queens ~ cover_type +
                           bombus_floral_abun_scaled +
                           num_burrows_scaled +
                           prop_inactive_scaled +
                           bee_season_type +
                           (1|plot_id),
                         ziformula = ~ cover_type +
                           bombus_floral_abun_scaled +
                           num_burrows_scaled +
                           prop_inactive_scaled +
```

```
                        bee_season_type +
                        (1|plot_id),
                      family = poisson,
                      data = field_data)
```

```
## Warning in finalizeTMB(TMBStruc, obj, fit, h, data.tmb.old): Model convergence
## problem; non-positive-definite Hessian matrix. See vignette('troubleshooting')
```

```
NSQ_siteOnly <- glmmTMB(nest_searching_queens ~ cover_type +
                        bombus_floral_abun_scaled +
                        num_burrows_scaled +
                        prop_inactive_scaled +
                        bee_season_type +
                        (1|site_id),
                      ziformula = ~ cover_type +
                        bombus_floral_abun_scaled +
                        num_burrows_scaled +
                        prop_inactive_scaled +
                        bee_season_type +
                        (1|site_id),
                      family = poisson,
                      data = field_data)

NSQ_AreaPlot <-  glmmTMB(nest_searching_queens ~ cover_type +
                        bombus_floral_abun_scaled +
                        num_burrows_scaled +
                        prop_inactive_scaled +
                        bee_season_type +
                        (1|area/plot_id),
                      ziformula = ~ cover_type +
                        bombus_floral_abun_scaled +
                        num_burrows_scaled +
                        prop_inactive_scaled +
                        bee_season_type +
                        (1|area/plot_id),
                      family = poisson,
                      data = field_data)
```

```
## Warning in finalizeTMB(TMBStruc, obj, fit, h, data.tmb.old): Model convergence
## problem; non-positive-definite Hessian matrix. See vignette('troubleshooting')
```

```
NSQ_noRE <- glmmTMB(nest_searching_queens ~ cover_type +
                      bombus_floral_abun_scaled +
                      num_burrows_scaled +
                      prop_inactive_scaled +
                      bee_season_type,
                    ziformula = ~ cover_type +
                      bombus_floral_abun_scaled +
                      num_burrows_scaled +
                      prop_inactive_scaled +
                      bee_season_type,  # zero-inflation model
                    family = poisson, #poisson
```

```
                  data = field_data)

#compare models using maximum likelihood test
anova(NSQ_full, NSQ_noArea, NSQ_plotOnly, NSQ_siteOnly, NSQ_AreaPlot, NSQ_noRE)
```

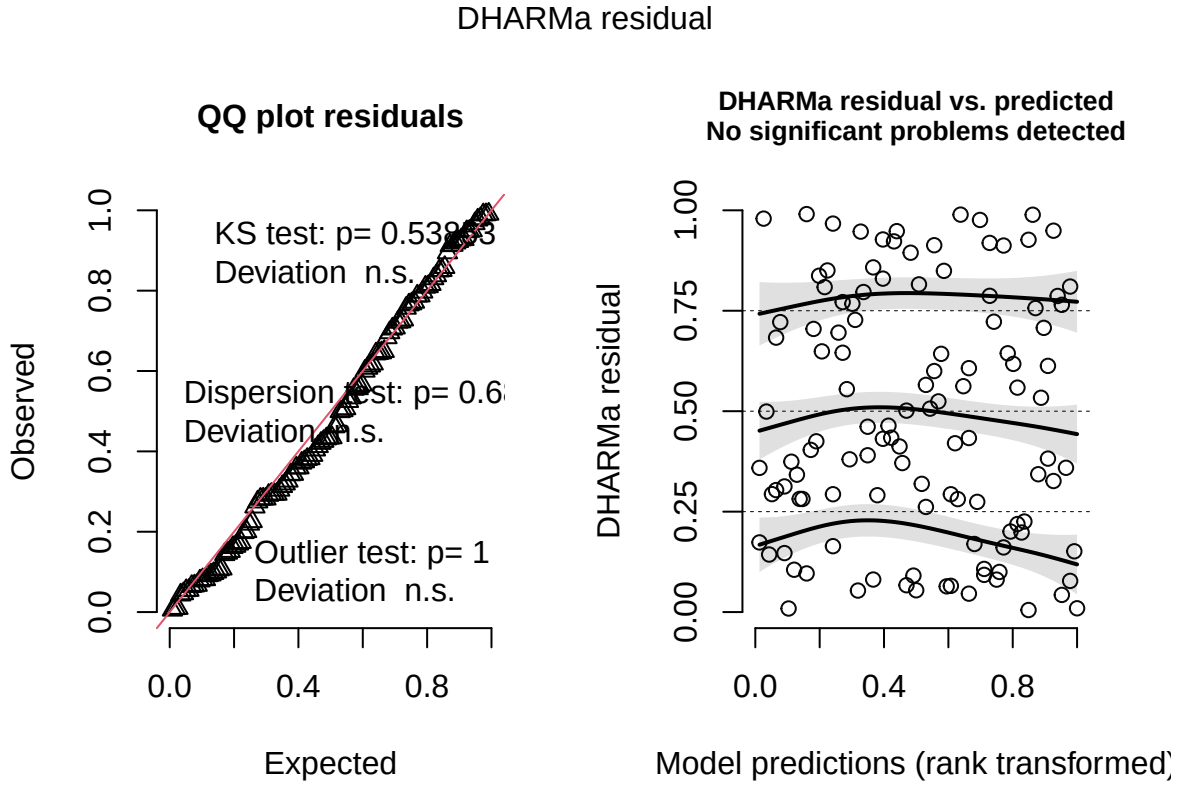
```
## Data: field_data
## Models:
## NSQ_noRE: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bombus
## NSQ_noRE:     num_burrows_scaled + prop_inactive_scaled + bee_season_type, zi=    prop_inactive_scal
## NSQ_plotOnly: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bo
## NSQ_plotOnly:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=    prop_inacti
## NSQ_plotOnly:     (1 | plot_id), zi=~cover_type + bombus_floral_abun_scaled + num_burrows_scaled + ,
## NSQ_siteOnly: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=    prop_inactiv
## NSQ_siteOnly:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=~cover_type + b
## NSQ_siteOnly:     (1 | site_id), zi=    prop_inactive_scaled + bee_season_type + (1 | site_id/plot_i
## NSQ_noArea: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bombu
## NSQ_noArea:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=    prop_inactive
## NSQ_noArea:     (1 | site_id/plot_id), zi=~cover_type + bombus_floral_abun_scaled + num_burrows_scal
## NSQ_AreaPlot: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=    prop_inactiv
## NSQ_AreaPlot:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=~cover_type + b
## NSQ_AreaPlot:     (1 | area/plot_id), zi=    prop_inactive_scaled + bee_season_type, disp=~1
## NSQ_full: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bombus
## NSQ_full:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=    prop_inactive_s
## NSQ_full:     (1 | area/site_id/plot_id), zi=~cover_type + bombus_floral_abun_scaled + num_burrows_s
##              Df    AIC    BIC  logLik deviance Chisq Chi Df Pr(>Chisq)
## NSQ_noRE     12 122.36 155.40 -49.178   98.356
## NSQ_plotOnly 14                                        2
## NSQ_siteOnly 14 126.36 164.91 -49.178   98.356         0
## NSQ_noArea   16                                        2
## NSQ_AreaPlot 16                                        0
## NSQ_full     18                                        2
```

```
##results tells us the model without random effects is best (lowest AIC).
```

2. Check diagnostics of best model.

- Looks good

```
residuals_nsq_noRE <- simulateResiduals(fittedModel =NSQ_noRE, plot = TRUE)
```



3. Look at results

```
summary(NSQ_noRE)
```

```
##  Family: poisson  ( log )
## Formula:
## nest_searching_queens ~ cover_type + bombus_floral_abun_scaled +
##     num_burrows_scaled + prop_inactive_scaled + bee_season_type
## Zero inflation:
## ~cover_type + bombus_floral_abun_scaled + num_burrows_scaled +
##     prop_inactive_scaled + bee_season_type
## Data: field_data
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     122.4     155.4     -49.2      98.4       104
##
##
## Conditional model:
##                            Estimate Std. Error z value Pr(>|z|)
## (Intercept)                 -0.5946     0.4351  -1.367   0.1717
## cover_typegrass              0.7957     0.4964   1.603   0.1090
## bombus_floral_abun_scaled   -0.1537     0.2124  -0.724   0.4691
## num_burrows_scaled           0.6810     0.3346   2.036   0.0418 *
## prop_inactive_scaled        -0.4053     0.2592  -1.564   0.1179
## bee_season_typeworker       -2.3731     0.7604  -3.121   0.0018 **
```

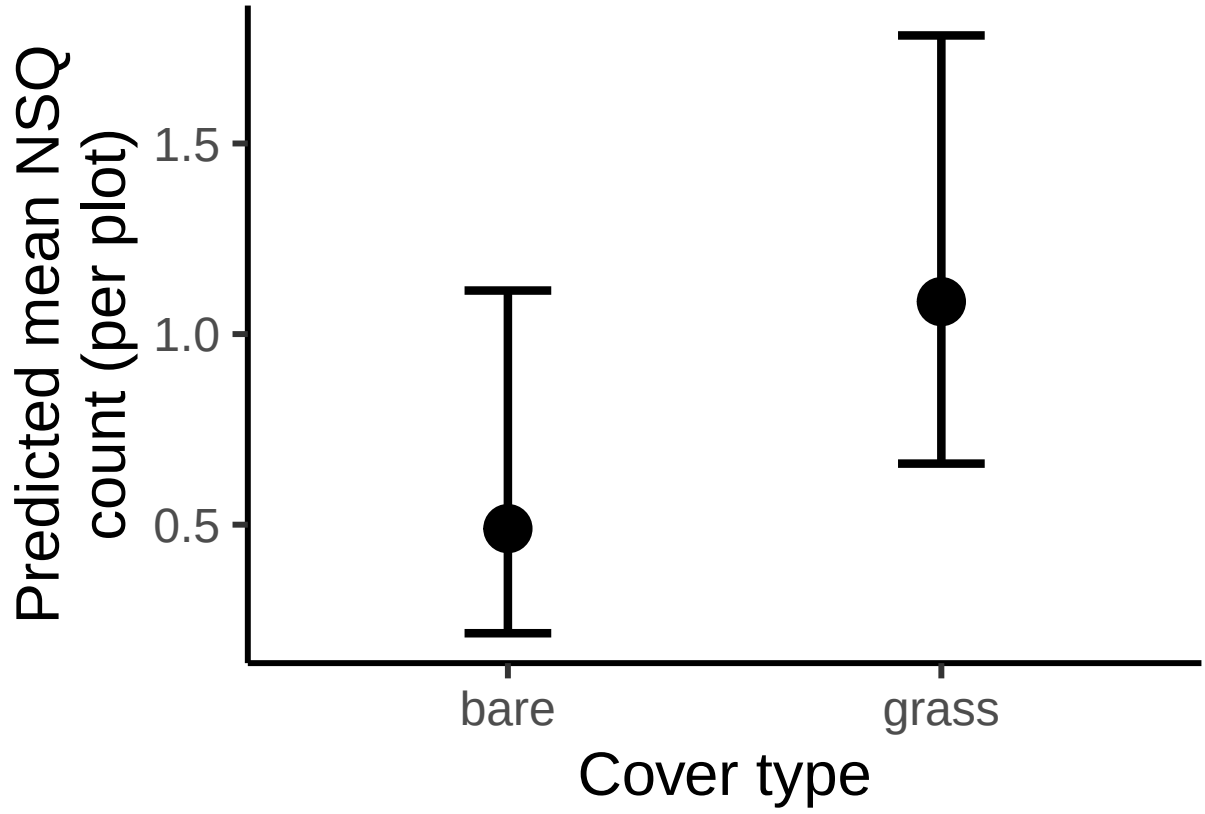
```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                            Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -7.340      6.223  -1.180    0.238
## cover_typegrass             -22.434     17.284  -1.298    0.194
## bombus_floral_abun_scaled    -8.652      6.776  -1.277    0.202
## num_burrows_scaled           25.258     18.300   1.380    0.168
## prop_inactive_scaled        -38.354     28.059  -1.367    0.172
## bee_season_typeworker         1.863      3.655   0.510    0.610
```

Plots

Number of nest-searching queens vs grass/forb

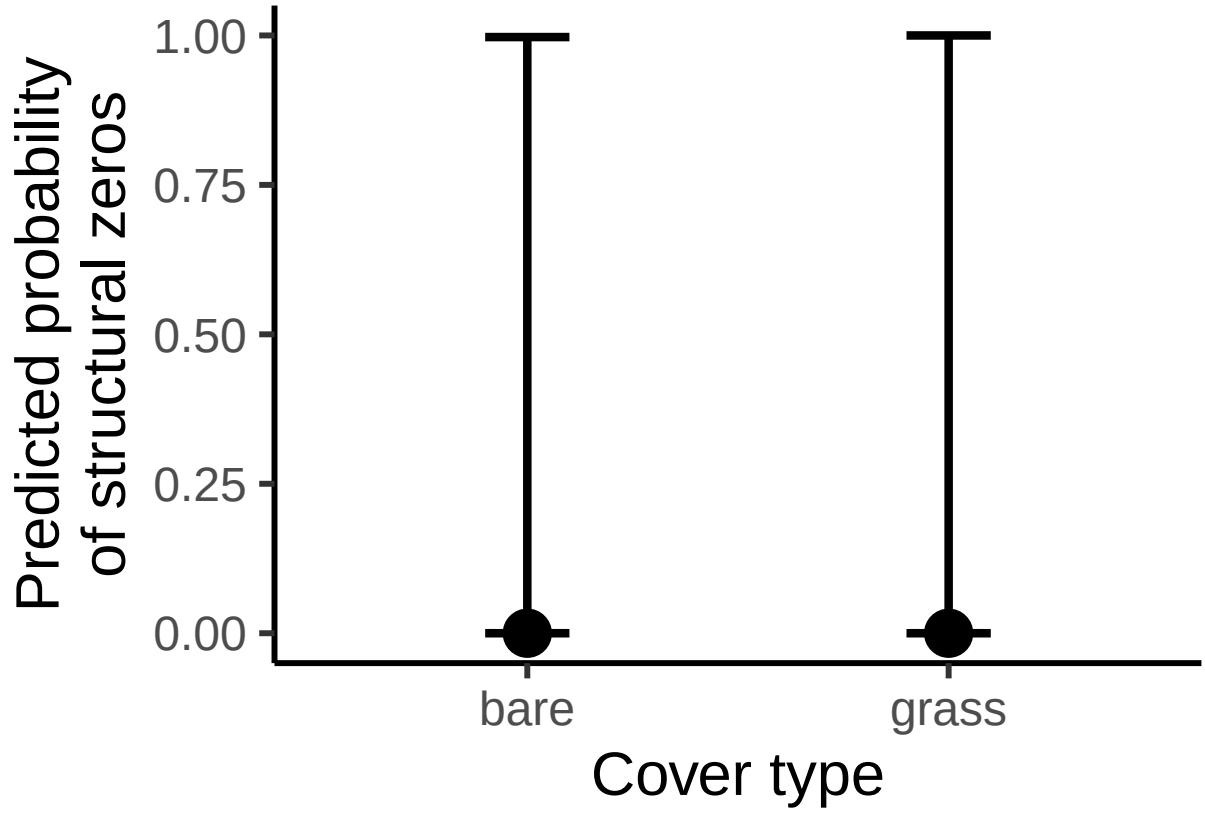
Conditional model:

```
#create new data grid
nsq_data_cover<- expand.grid(
  cover_type = unique(field_data$cover_type),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching",
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE)
)

# Predictions for conditional model (count)
pred_nsq_cover_cond <- predict(NSQ_noRE,
                               newdata = nsq_data_cover,
                               type = "link",
                               se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_cover_df <- cbind(nsq_data_cover,
                  cond_fit_log = pred_nsq_cover_cond$fit,
                  cond_se_log = pred_nsq_cover_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_cover_df <- pred_nsq_cover_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

#plot
ggplot(pred_nsq_cover_df, aes(x = cover_type, y = cond_fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(
       x = "Cover type",
       y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Zero-inflated model:

```
# Predictions for ZI model
pred_nsq_cover_zi <- predict(NSQ_noRE,
                             newdata = nsq_data_cover,
                             type = "zlink",
                            se.fit = TRUE)

# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_cover_df <- cbind(
  nsq_data_cover,
  zi_fit = plogis(pred_nsq_cover_zi$fit),
  zi_lower = plogis(pred_nsq_cover_zi$fit - 1.96 * pred_nsq_cover_zi$se.fit),
  zi_upper = plogis(pred_nsq_cover_zi$fit + 1.96 * pred_nsq_cover_zi$se.fit)
)

# Plot zi predictions
ggplot(pred_nsq_cover_df, aes(x = cover_type, y = zi_fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = zi_lower, ymax = zi_upper), width = 0.2, position = position_dodge(0.6), lin
  labs( x = "Cover type", y = "Predicted probability\nof structural zeros") +
  theme_classic(base_size = 23)
```

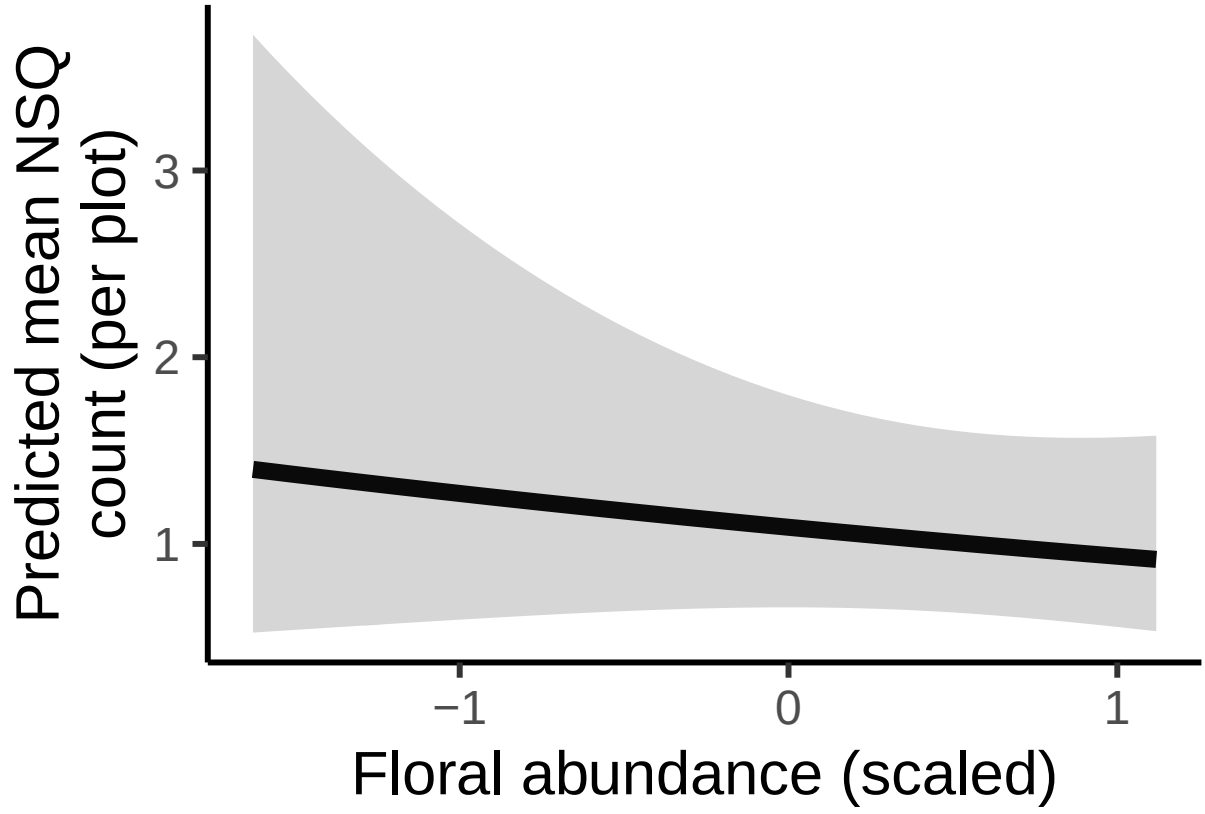


Number of nest-searching queens vs floral abundance

Conditional model:

```
# Define a sequence of values for bombus_floral_abun_scaled across its observed range
floral_seq <- seq(
  from = min(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  to = max(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  length.out = 50
)

# Create data grid over bombus_floral_abun_scaled
nsq_data_floral <- data.frame(
  bombus_floral_abun_scaled = floral_seq,
  cover_type = "grass",
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching"
)

# Predictions for conditional model (count)
pred_nsq_floral_cond <- predict(NSQ_noRE,
                                newdata = nsq_data_floral,
                                type = "link",
                                se.fit = TRUE)

# Combine predictions with newdata
```

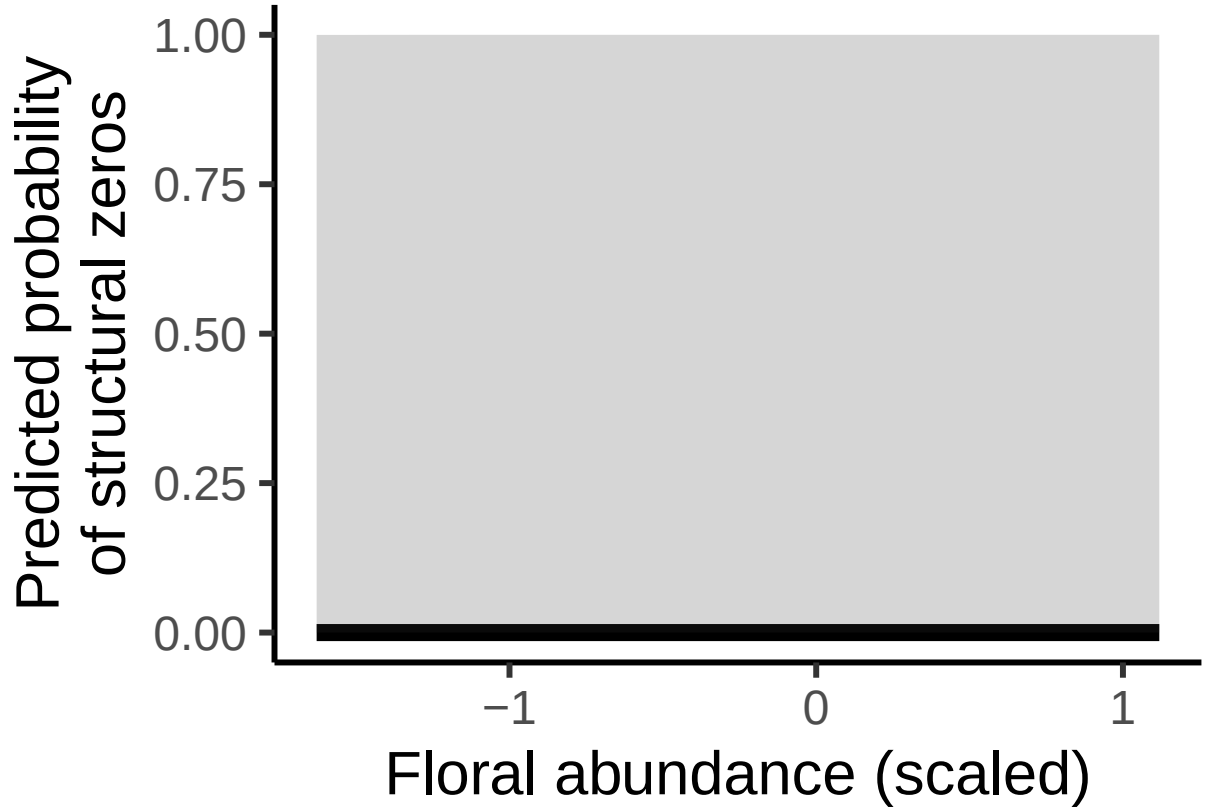
```r
pred_nsq_floral_df <- cbind(nsq_data_floral,
                 cond_fit_log = pred_nsq_floral_cond$fit,
                 cond_se_log = pred_nsq_floral_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_floral_df <- pred_nsq_floral_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

# Plot conditional predictions
ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = cond_fit)) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper), alpha = 0.2) +
  labs( x = "Floral abundance (scaled)" , y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Zero-inflated model:

```r
# Predictions for ZI model
pred_nsq_floral_zi <- predict(NSQ_noRE,
                              newdata = nsq_data_floral,
                              type = "zlink",
                             se.fit = TRUE)
```

```r
# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_floral_df <- cbind(
  nsq_data_floral,
  zi_fit = plogis(pred_nsq_floral_zi$fit),
  zi_lower = plogis(pred_nsq_floral_zi$fit - 1.96 * pred_nsq_floral_zi$se.fit),
  zi_upper = plogis(pred_nsq_floral_zi$fit + 1.96 * pred_nsq_floral_zi$se.fit)
)

# Plot zi predictions
ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = zi_fit)) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = zi_lower, ymax = zi_upper), alpha = 0.2) +
  labs(x = "Floral abundance (scaled)", y = "Predicted probability\nof structural zeros") +
  theme_classic(base_size = 23)
```

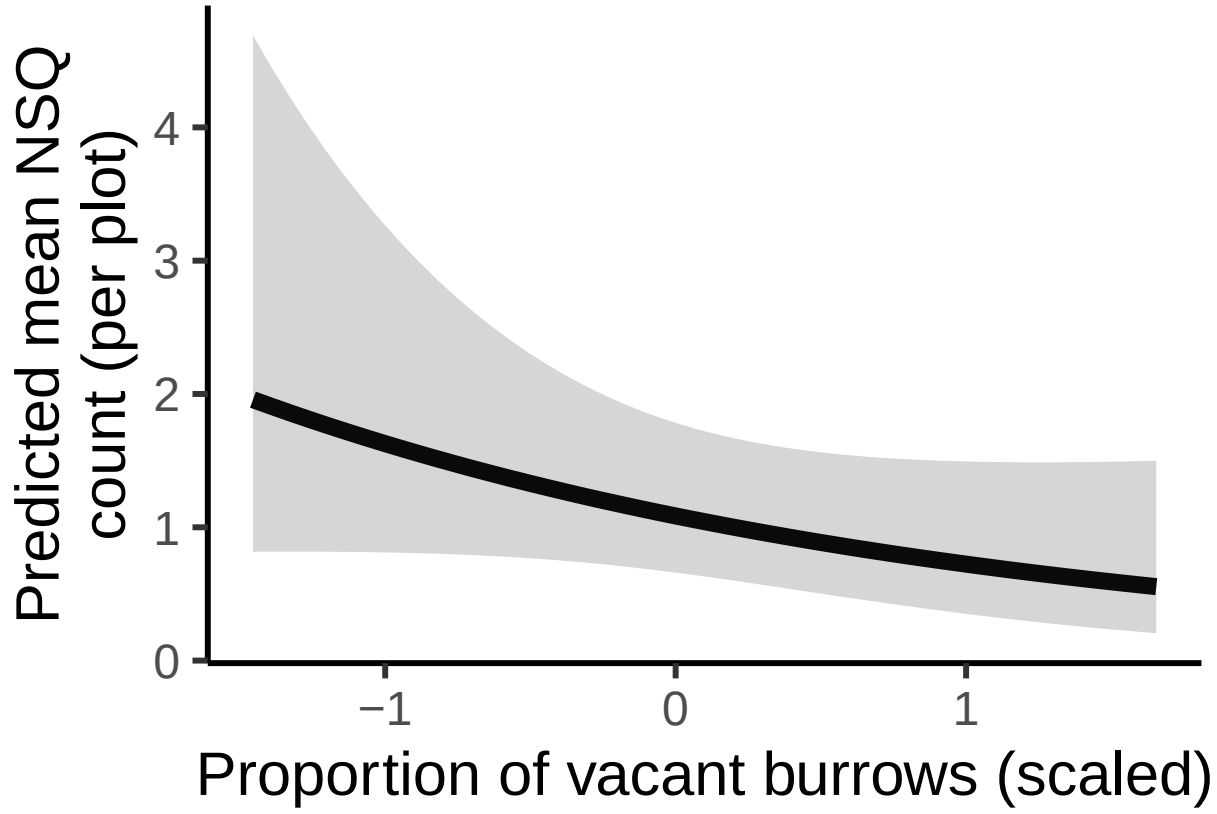


Number of nest-searching queens vs burrow vacancy proportion

Conditional model:

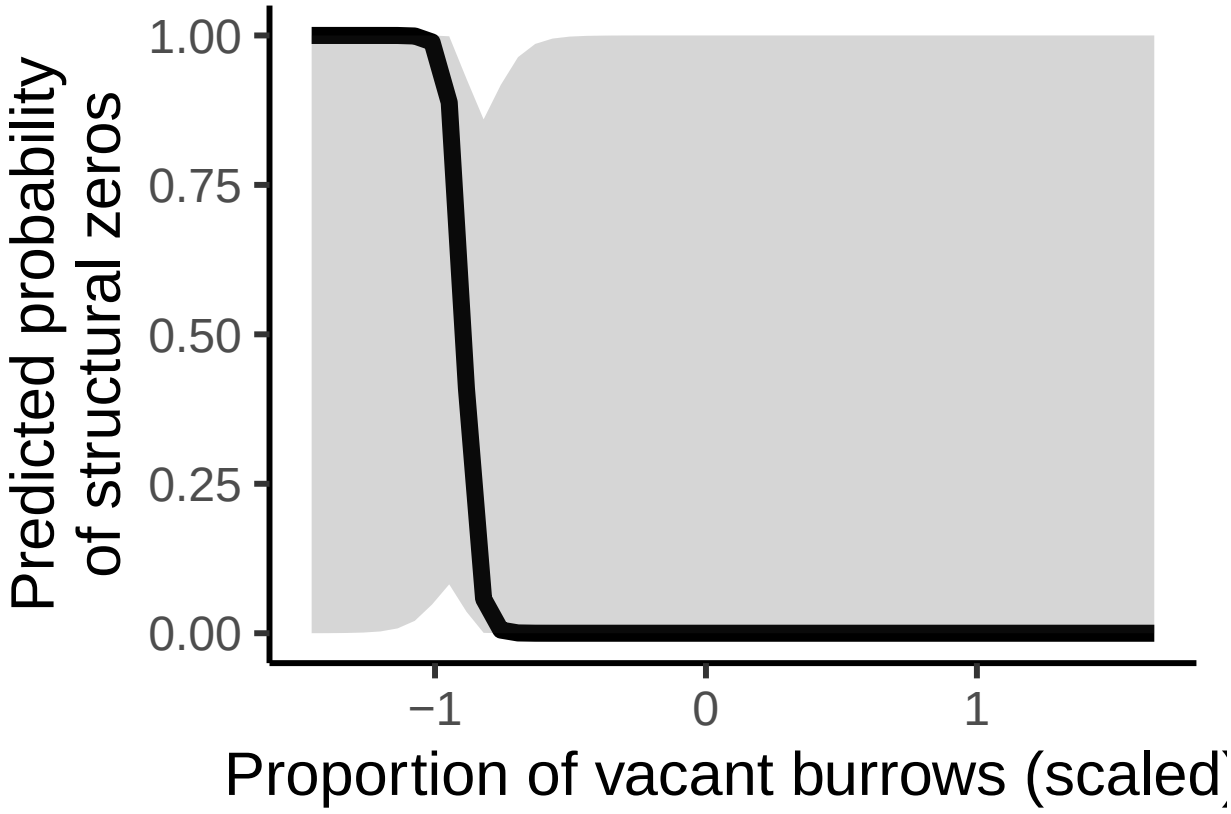
```r
#create new data grid
nsq_data_vacancy<- expand.grid(
  cover_type = "grass",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching",
  prop_inactive_scaled = seq(min(field_data$prop_inactive_scaled, na.rm = TRUE),
```

```
                              max(field_data$prop_inactive_scaled, na.rm = TRUE),
                              length.out = 50)
)

# Predictions for conditional model (count)
pred_nsq_vacancy_cond <- predict(NSQ_noRE,
                              newdata = nsq_data_vacancy,
                              type = "link",
                              se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_vacancy_df <- cbind(nsq_data_vacancy,
                  cond_fit_log = pred_nsq_vacancy_cond$fit,
                  cond_se_log = pred_nsq_vacancy_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_vacancy_df <- pred_nsq_vacancy_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

#plot
ggplot(pred_nsq_vacancy_df, aes(x = prop_inactive_scaled, y = cond_fit)) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper), alpha = 0.2) +
  labs(
       x = "Proportion of vacant burrows (scaled)",
       y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Zero-inflated model:

```
# Predictions for ZI model
pred_nsq_vacancy_zi <- predict(NSQ_noRE,
                               newdata = nsq_data_vacancy,
                               type = "zlink",
                              se.fit = TRUE)

# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_vacancy_df <- cbind(
  nsq_data_vacancy,
  zi_fit = plogis(pred_nsq_vacancy_zi$fit),
  zi_lower = plogis(pred_nsq_vacancy_zi$fit - 1.96 * pred_nsq_vacancy_zi$se.fit),
  zi_upper = plogis(pred_nsq_vacancy_zi$fit + 1.96 * pred_nsq_vacancy_zi$se.fit)
)

# Plot zi predictions
ggplot(pred_nsq_vacancy_df, aes(x = prop_inactive_scaled, y = zi_fit)) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = zi_lower, ymax = zi_upper), alpha = 0.2) +
  labs( x = "Proportion of vacant burrows (scaled)", y = "Predicted probability\nof structural zeros") +
  theme_classic(base_size = 23)
```



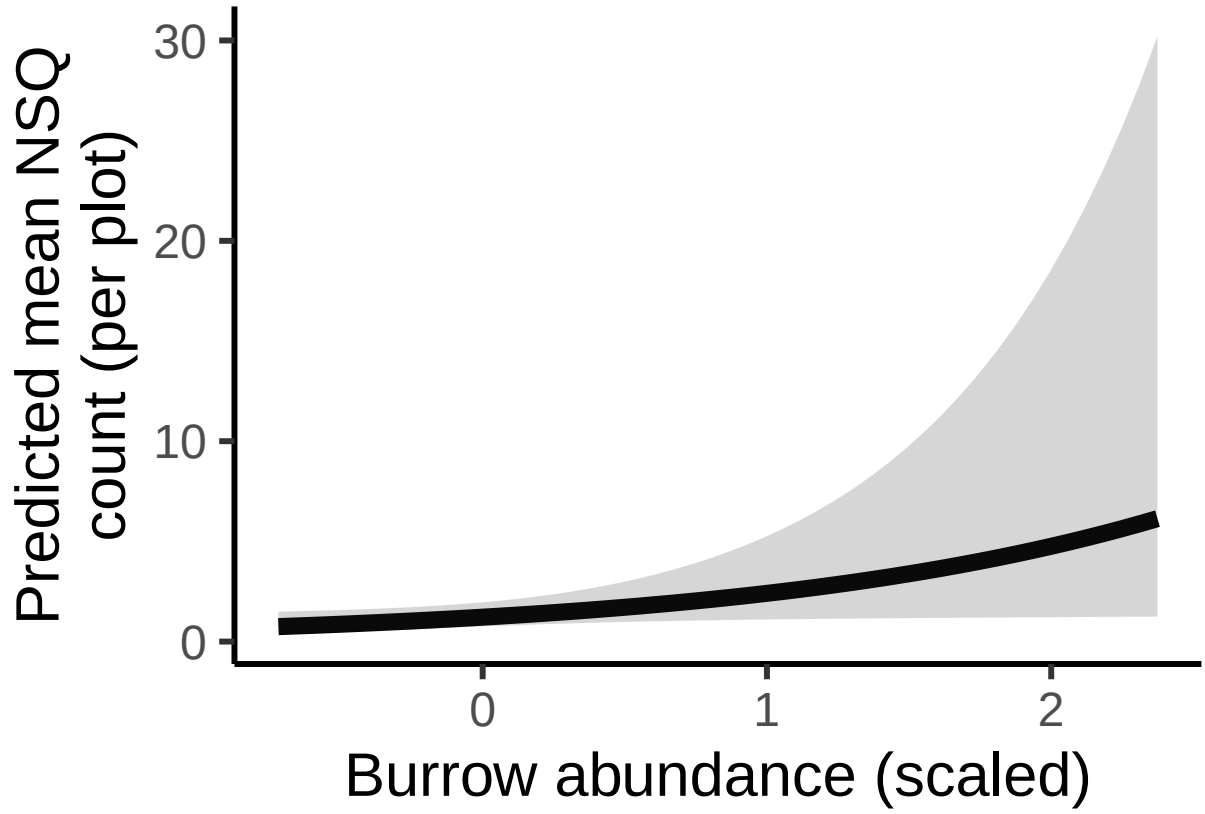
Number of nest-searching queens vs number of rodent burrows

Conditional model:

```
# Create data grid over rodent_burrows_scaled
nsq_data_rodent <- data.frame(
  num_burrows_scaled = seq(min(field_data$num_burrows_scaled), max(field_data$num_burrows_scaled), leng
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  cover_type = "grass",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  bee_season_type = "nestsearching"
)

# Predictions for conditional model (count)
pred_nsq_rodent_cond <- predict(
  NSQ_noRE,
  newdata = nsq_data_rodent,
  type = "link", ##log-scale
  se.fit = TRUE)

# Combine predictions with newdata
pred_nsq_rodent_df <- cbind(nsq_data_rodent,
                cond_fit_log = pred_nsq_rodent_cond$fit,
                cond_se_log = pred_nsq_rodent_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_rodent_df <- pred_nsq_rodent_df %>%
```

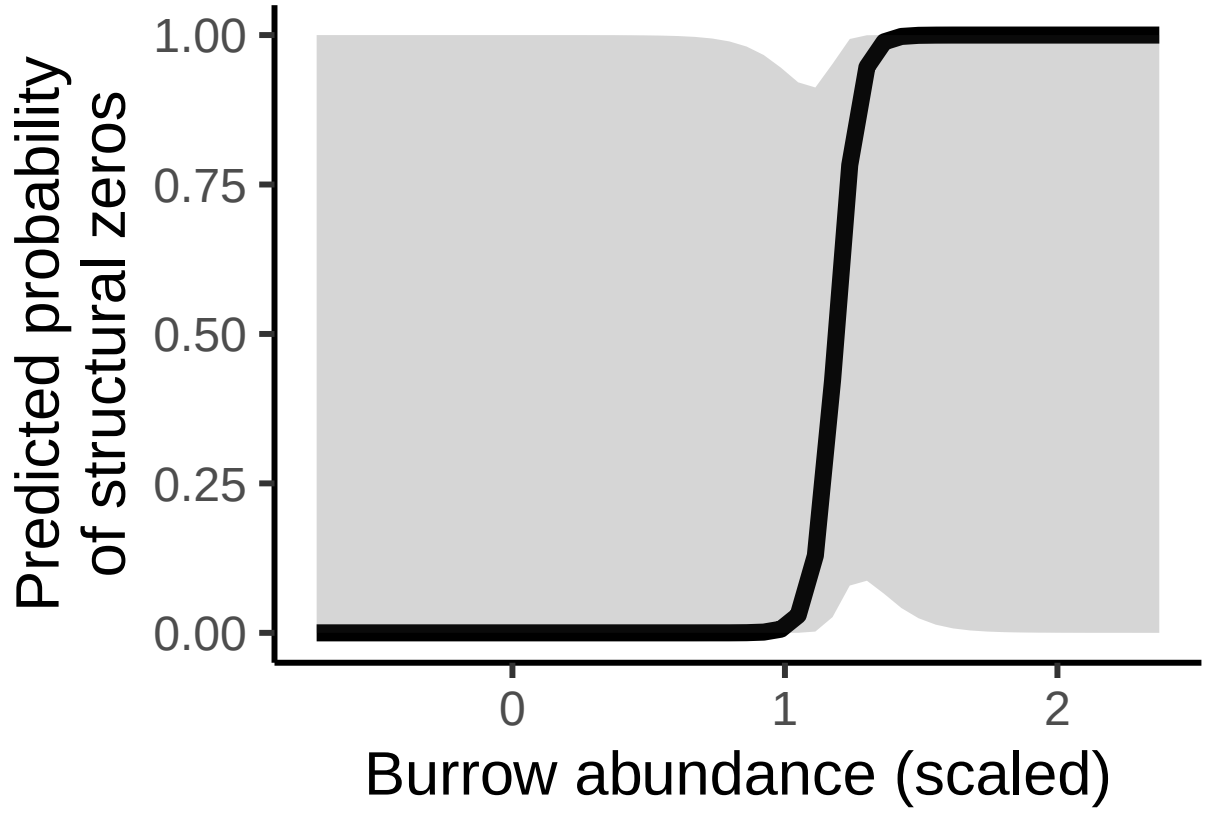
```
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

ggplot(pred_nsq_rodent_df, aes(x = num_burrows_scaled, y = cond_fit)) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper), alpha = 0.2) +
  labs(
    x = "Burrow abundance (scaled)",
    y = "Predicted mean NSQ\ncount (per plot)"
  ) +
  theme_classic(base_size = 23)
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



Zero-inflated model:

```r
# Predictions for ZI model
pred_nsq_rodent_zi <- predict(NSQ_noRE,
                              newdata = nsq_data_rodent,
                              type = "zlink",
                             se.fit = TRUE)

# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_rodent_df <- cbind(
  nsq_data_rodent,
  zi_fit = plogis(pred_nsq_rodent_zi$fit),
  zi_lower = plogis(pred_nsq_rodent_zi$fit - 1.96 * pred_nsq_rodent_zi$se.fit),
  zi_upper = plogis(pred_nsq_rodent_zi$fit + 1.96 * pred_nsq_rodent_zi$se.fit)
)

ggplot(pred_nsq_rodent_df, aes(x = num_burrows_scaled, y = zi_fit)) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = zi_lower, ymax = zi_upper), alpha = 0.2) +
  labs(x = "Burrow abundance (scaled)", y = "Predicted probability\nof structural zeros") +
  theme_classic(base_size = 23)
```

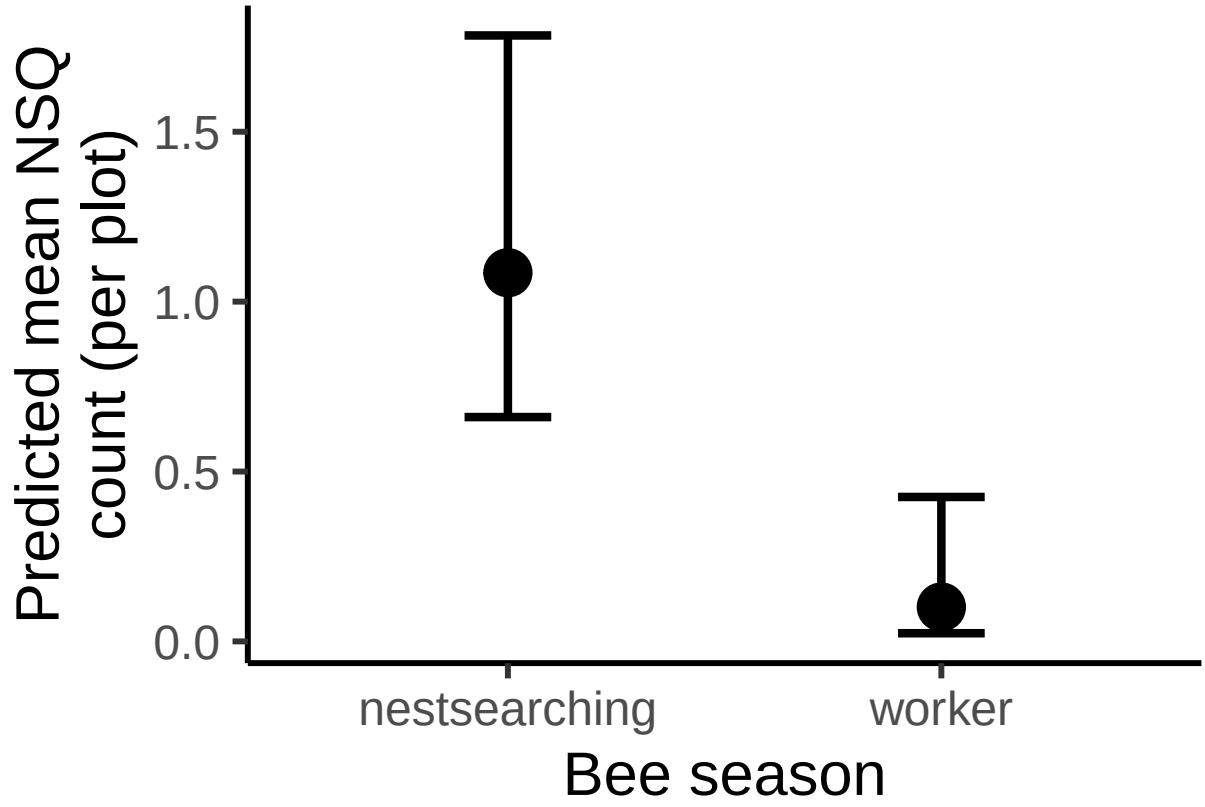


Number of nest-searching queens vs season type

Conditional model:

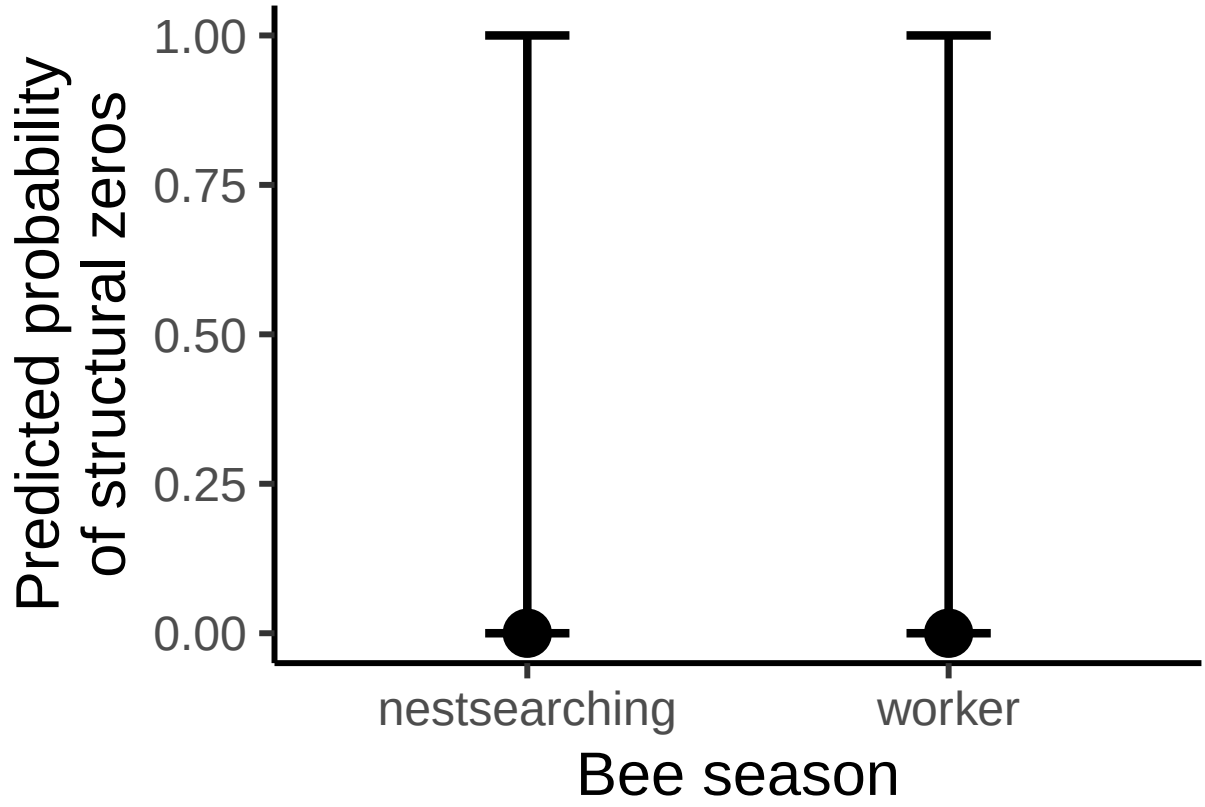
```r
# Create newdata with discrete factor levels for bee_season_type
nsq_data_season <- data.frame(
```

```
  cover_type = "grass",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = factor(c("nestsearching", "worker"), levels = levels(field_data$bee_season_type))
)

# Predictions for conditional model (count)
pred_nsq_season_cond <- predict(
  NSQ_noRE,
  newdata = nsq_data_season,
  type = "link", ##log-scale
  se.fit = TRUE)

# Combine predictions with newdata
pred_nsq_season_df <- cbind(nsq_data_season,
                 cond_fit_log = pred_nsq_season_cond$fit,
                 cond_se_log = pred_nsq_season_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_season_df <- pred_nsq_season_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

# Plot conditional predictions
ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = cond_fit)) +
  geom_point(size=8) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(x = "Bee season", y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Zero-inflated model:

```
# Predictions for ZI model
pred_nsq_season_zi <- predict(NSQ_noRE,
                              newdata = nsq_data_season,
                              type = "zlink",
                             se.fit = TRUE)

# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_season_df <- cbind(
  nsq_data_season,
  zi_fit = plogis(pred_nsq_season_zi$fit),
  zi_lower = plogis(pred_nsq_season_zi$fit - 1.96 * pred_nsq_season_zi$se.fit),
  zi_upper = plogis(pred_nsq_season_zi$fit + 1.96 * pred_nsq_season_zi$se.fit)
)

ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = zi_fit)) +
  geom_point(size=8) +
  geom_errorbar(aes(ymin = zi_lower, ymax = zi_upper), width = 0.2, position = position_dodge(0.6), lin
  labs(x = "Bee season", y = "Predicted probability\nof structural zeros") +
  theme_classic(base_size = 23)
```



Findings

In conditional model (modelling abundance of nest-searching queens):

- no significant effect of grass+forb%, floral abundance, bee season
- significant positive effect of number of burrows

In zero-inflation model (modelling presence/absence of nest-searching queens):

- no significant effect of floral abundance
- Significant effect of grass+forb% (less likely to be absent with more grass/forb), number of burrows (more likely to be absent with more burrows), bee season (more likely to be absent during worker season than nest-searching season).

Interestingly, the number of rodent burrows is associated with more bombus nest-searching in the conditional model, but the number of rodent burrows is associated with more structural zeros in the zero-inflation model. This means that if a site is suitable for nesting, more rodent burrows is associated with more bombus nest-searching. However, in some cases, rodent burrows may signal inhospitable habitat conditions for bumble bees.

Number of workers vs nest-searching queens, floral abundance, rodent burrows.

Go through similar model selection steps as for the rodent burrow model.

Steps

1) Check different random effects structures

- best fit = 1|site_id/plot_id –> **area not included**

```
workers_full <- glmmTMB(workers ~ nest_searching_queens_scaled +
                          bombus_floral_abun_scaled +
                          num_burrows_scaled +
                            prop_inactive_scaled +
                          bee_season_type +
                            (1|area/site_id/plot_id),
                        family = poisson,
                        data = field_data
                          )


workers_noArea <- glmmTMB(workers ~ nest_searching_queens_scaled +
                          bombus_floral_abun_scaled +
                          num_burrows_scaled +
                            prop_inactive_scaled +
                          bee_season_type+
                            (1|site_id/plot_id),
                        family = poisson,
                        data = field_data
                          )

workers_plotOnly <-  glmmTMB(workers ~ nest_searching_queens_scaled +
                          bombus_floral_abun_scaled +
                          num_burrows_scaled +
                            prop_inactive_scaled +
                          bee_season_type+
                            (1|plot_id),
                        family = poisson,
                        data = field_data
                          )

workers_siteOnly <- glmmTMB(workers ~ nest_searching_queens_scaled +
                          bombus_floral_abun_scaled +
                          num_burrows_scaled +
                            prop_inactive_scaled +
                          bee_season_type+
                            (1|site_id),
                        family = poisson,
                        data = field_data
                          )
```

```r
workers_AreaPlot <-  glmmTMB(workers ~ nest_searching_queens_scaled +
                       bombus_floral_abun_scaled +
                       num_burrows_scaled +
                         prop_inactive_scaled +
                       bee_season_type+
                         (1|area/plot_id),
                     family = poisson,
                     data = field_data
                       )

workers_noRE <- glmmTMB(workers ~ nest_searching_queens_scaled +
                       bombus_floral_abun_scaled +
                       num_burrows_scaled +
                         prop_inactive_scaled +
                       bee_season_type,
                     family = poisson,
                     data = field_data
                       )

#compare models using maximum likelihood test
anova(workers_full, workers_noArea, workers_plotOnly, workers_siteOnly, workers_AreaPlot, workers_noRE)
```

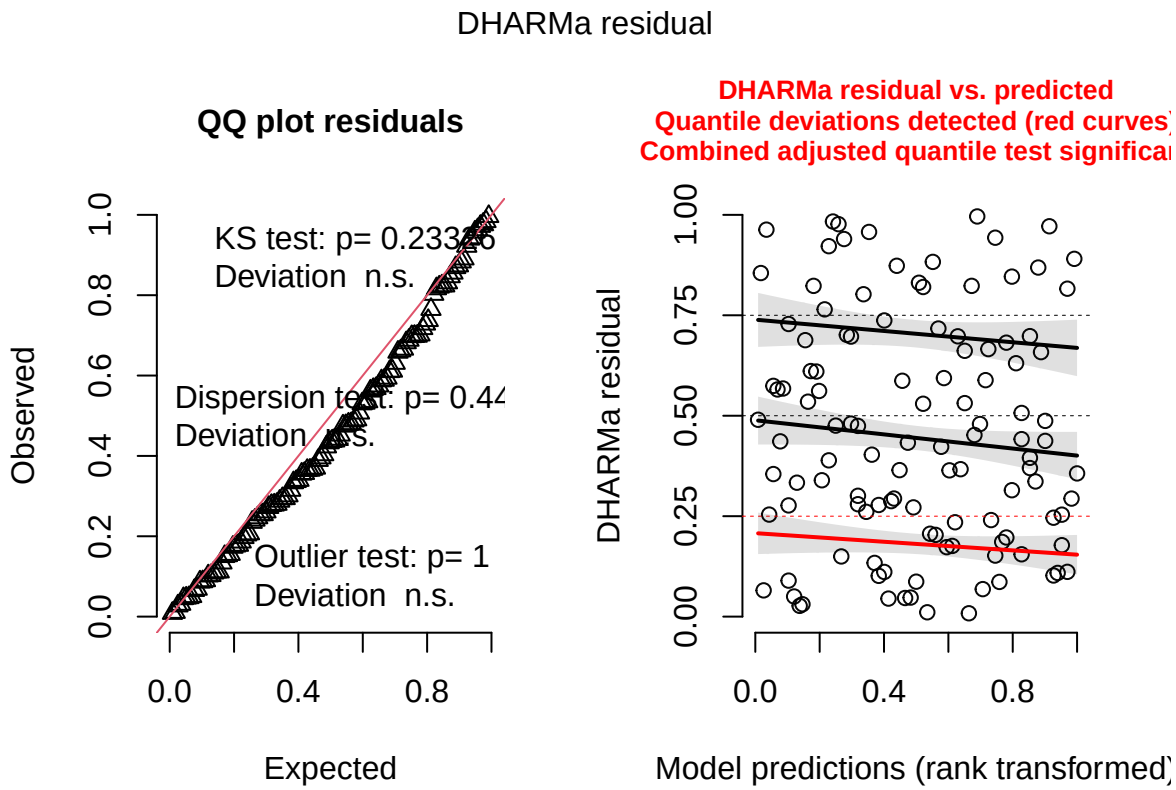
```
## Data: field_data
## Models:
## workers_noRE: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_noRE:     num_burrows_scaled + prop_inactive_scaled + bee_season_type, zi=~0, disp=~1
## workers_plotOnly: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=
## workers_plotOnly:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=~0, disp=~1
## workers_plotOnly:     (1 | plot_id), zi=~0, disp=~1
## workers_siteOnly: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=
## workers_siteOnly:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=~0, disp=~1
## workers_siteOnly:     (1 | site_id), zi=~0, disp=~1
## workers_noArea: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~
## workers_noArea:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=~0, disp=~1
## workers_noArea:     (1 | site_id/plot_id), zi=~0, disp=~1
## workers_AreaPlot: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=
## workers_AreaPlot:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=~0, disp=~1
## workers_AreaPlot:     (1 | area/plot_id), zi=~0, disp=~1
## workers_full: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_full:     num_burrows_scaled + prop_inactive_scaled + bee_season_type + , zi=~0, disp=~1
## workers_full:     (1 | area/site_id/plot_id), zi=~0, disp=~1
##                  Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
## workers_noRE      6 175.50 192.02 -81.748   163.50
## workers_plotOnly  7 173.90 193.18 -79.952   159.90 3.5910      1    0.05809 .
## workers_siteOnly  7 165.67 184.95 -75.836   151.67 8.2316      0    < 2e-16 ***
## workers_noArea    8 164.91 186.94 -74.455   148.91 2.7629      1    0.09647 .
## workers_AreaPlot  8 168.75 190.78 -76.374   152.75 0.0000      0    1.00000
## workers_full      9 166.91 191.69 -74.455   148.91 3.8374      1    0.05012 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##results tells us the model with no area is best (lowest AIC).
```

2. Check diagnostics of best model.

- No significant diagnostic tests. Good to go.

```
#simulate residuals
residuals_workers_noArea <- simulateResiduals(fittedModel = workers_noArea, plot = TRUE)
```



3. Check results of final model

```
summary(workers_noArea)
```

```
##  Family: poisson  ( log )
## Formula:
## workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled +
##     num_burrows_scaled + prop_inactive_scaled + bee_season_type +
##     (1 | site_id/plot_id)
## Data: field_data
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     164.9     186.9     -74.5     148.9       108
##
```

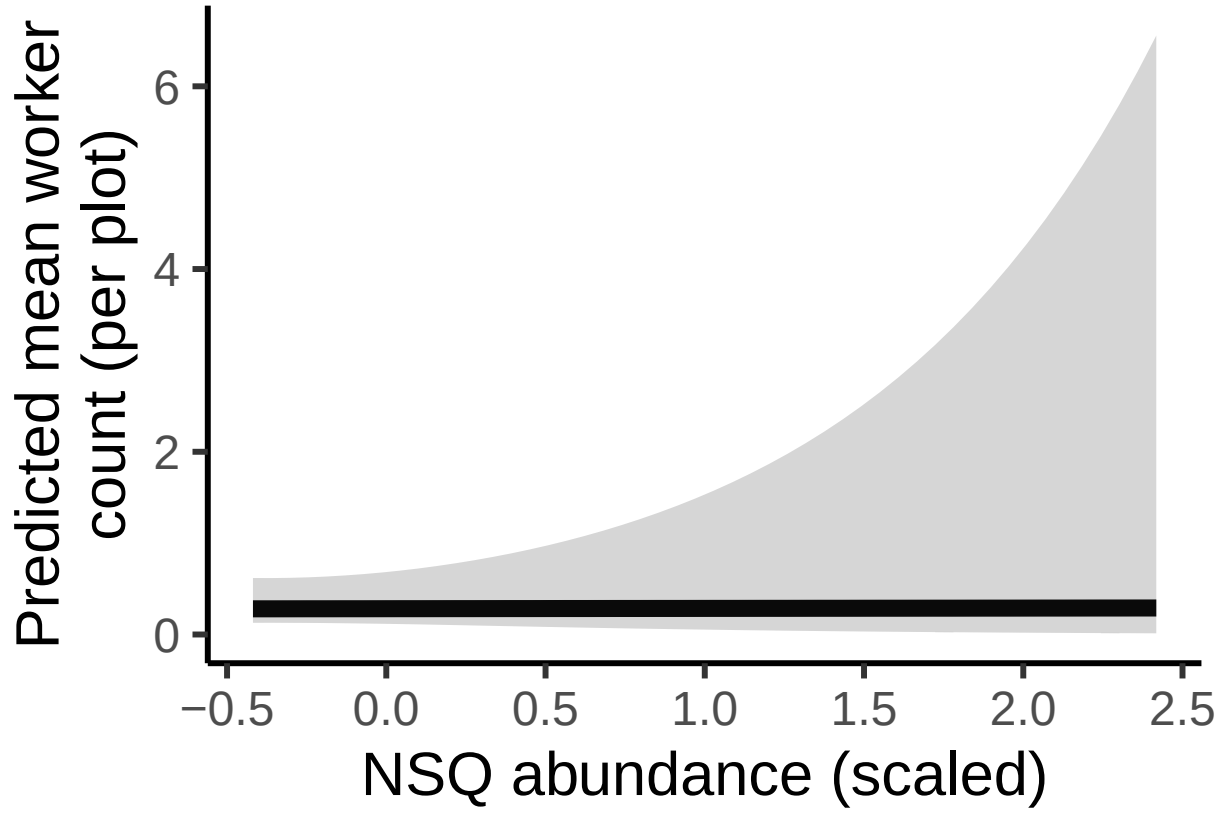
```
## Random effects:
##
## Conditional model:
##  Groups          Name        Variance Std.Dev.
##  plot_id:site_id (Intercept) 0.7704   0.8777
##  site_id         (Intercept) 0.2976   0.5455
## Number of obs: 116, groups:  plot_id:site_id, 20; site_id, 11
##
## Conditional model:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -3.56408    0.65332  -5.455 4.89e-08 ***
## nest_searching_queens_scaled  0.01148    0.55155   0.021 0.983388
## bombus_floral_abun_scaled     0.71632    0.22172   3.231 0.001235 **
## num_burrows_scaled            0.31621    0.37307   0.848 0.396675
## prop_inactive_scaled         -0.18788    0.35360  -0.531 0.595182
## bee_season_typeworker         2.33515    0.61408   3.803 0.000143 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

Number of workers vs number of nest-searching queens

```
# Define a sequence of values for nsq across its observed range
queen_seq <- seq(
  from = min(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  to = max(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data grid
worker_data_nsq<- expand.grid(
  nest_searching_queens_scaled = queen_seq,
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_nsq <- predict(workers_noArea,
                           newdata = worker_data_nsq,
                          type = "link",
                           se.fit = TRUE,
                           re.form = NA)

# Combine predictions with data frame
pred_worker_nsq_df <- cbind(worker_data_nsq,
                fit_log = pred_worker_nsq$fit,
                se_log = pred_worker_nsq$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_nsq_df <- pred_worker_nsq_df %>%
```

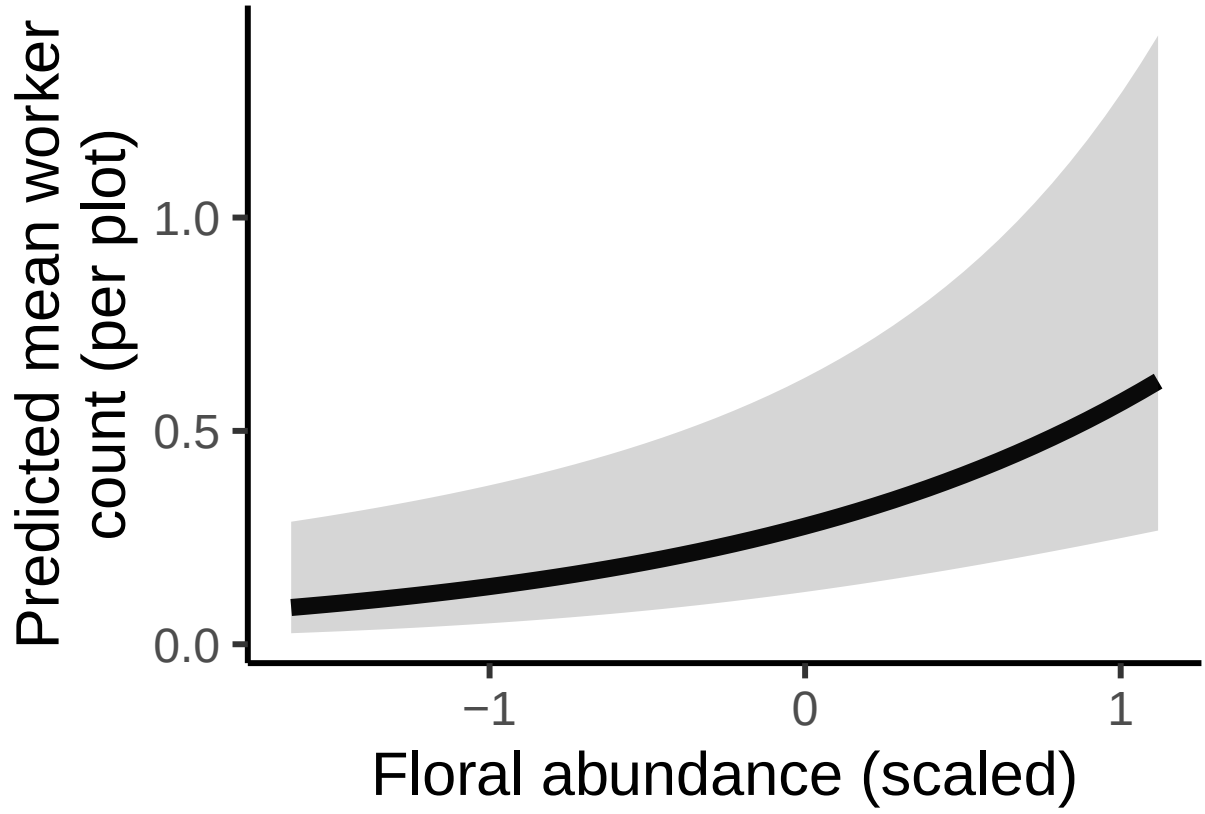
```
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

ggplot(pred_worker_nsq_df, aes(x = nest_searching_queens_scaled, y = fit)) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
       x = "NSQ abundance (scaled)",
       y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Number of workers vs floral abundance

```
#create new data grid
worker_data_floral<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = floral_seq,
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
```

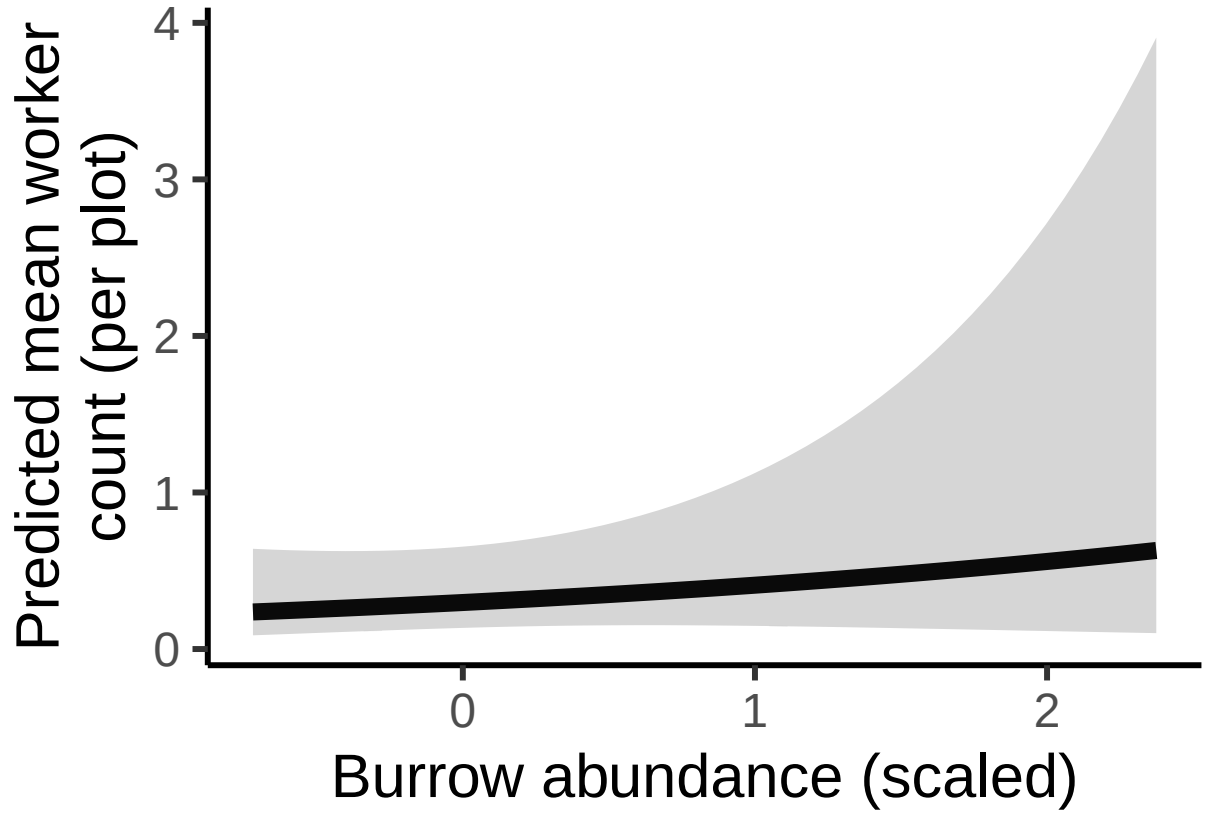
```r
pred_worker_floral <- predict(workers_noArea,
                              newdata = worker_data_floral,
                              type = "link",
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_worker_floral_df <- cbind(worker_data_floral,
                 fit_log = pred_worker_floral$fit,
                 se_log = pred_worker_floral$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_floral_df <- pred_worker_floral_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

ggplot(pred_worker_floral_df, aes(x = bombus_floral_abun_scaled, y = fit)) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
       x = "Floral abundance (scaled)",
       y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Number of workers vs number of rodent burrows

```
# Define a sequence of values for rodent burrows across its observed range
rodent_seq <- seq(
  from = min(field_data$num_burrows_scaled, na.rm = TRUE),
  to = max(field_data$num_burrows_scaled, na.rm = TRUE),
  length.out = 50)

#create new data grid
worker_data_rodent<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled =rodent_seq,
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_rodent <- predict(workers_noArea,
                              newdata = worker_data_rodent,
                              type = "link",
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_worker_rodent_df <- cbind(worker_data_rodent,
```

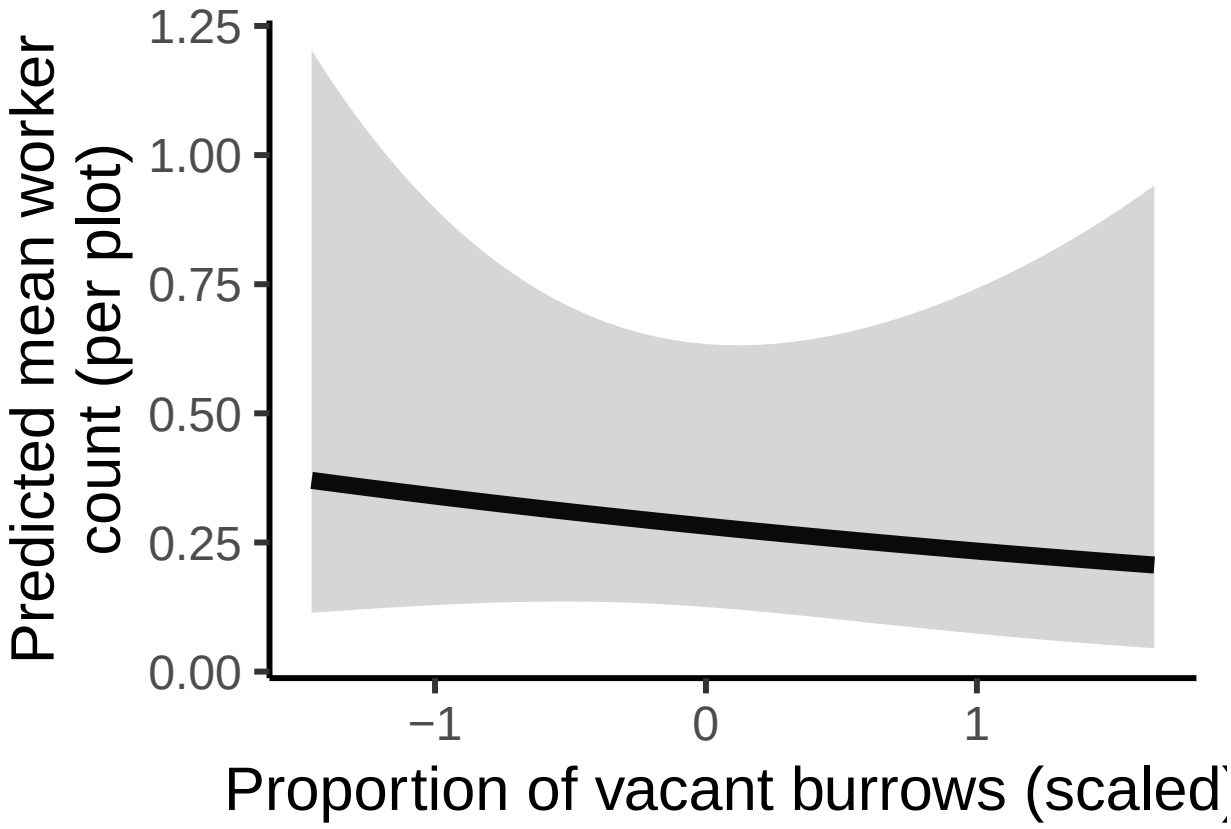
```
                fit_log = pred_worker_rodent$fit,
                se_log = pred_worker_rodent$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_rodent_df <- pred_worker_rodent_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

ggplot(pred_worker_rodent_df, aes(x = num_burrows_scaled, y = fit)) +
  geom_line(size = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
       x = "Burrow abundance (scaled)",
       y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Number of workers vs burrow vacancy proportion

```
#create new data grid
worker_data_vacancy <- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled =mean(field_data$num_burrows_scaled, na.rm = TRUE),
```

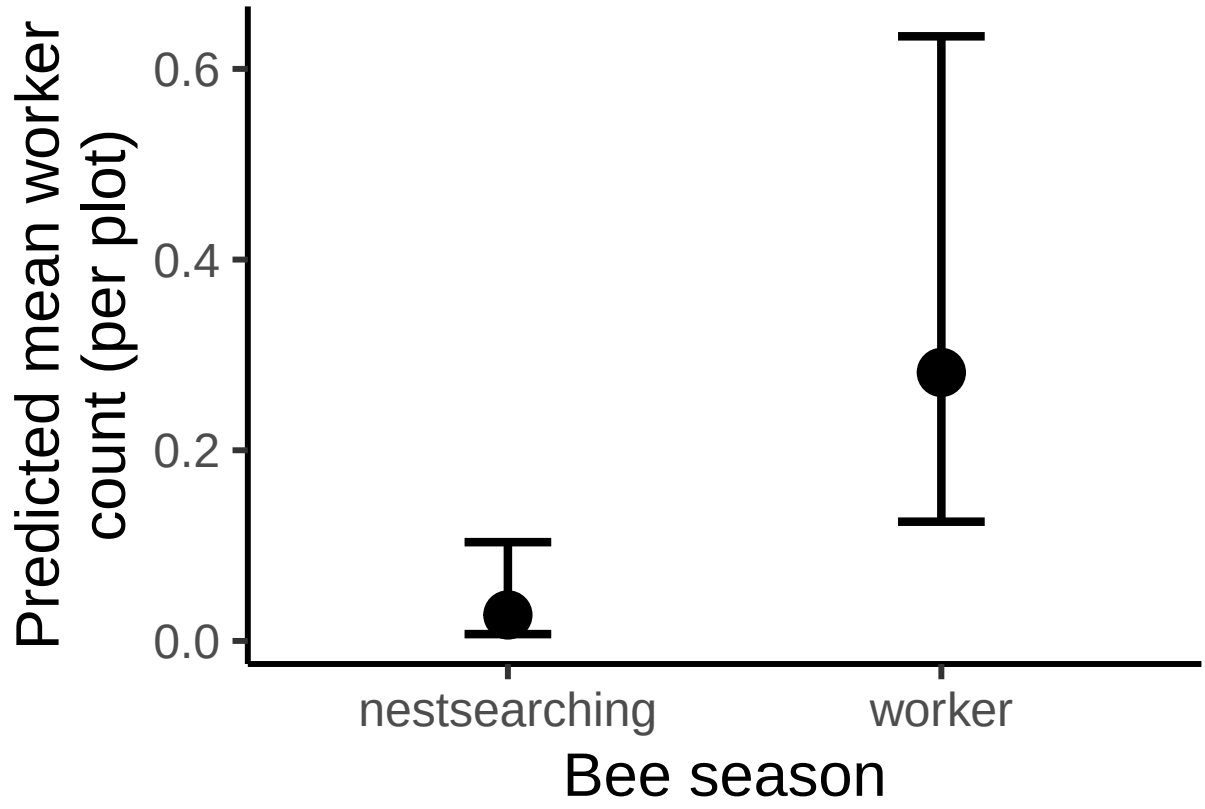
```
  bee_season_type = "worker",
  prop_inactive_scaled = seq(min(field_data$prop_inactive_scaled, na.rm = TRUE),
                             max(field_data$prop_inactive_scaled, na.rm = TRUE),
                             length.out = 50)
)

# Predictions
pred_worker_vacancy <- predict(workers_noArea,
                               newdata = worker_data_vacancy,
                               type = "link",
                               se.fit = TRUE,
                               re.form = NA
                               )

# Combine predictions with data frame
pred_worker_vacancy_df <- cbind(worker_data_vacancy,
                  fit_log = pred_worker_vacancy$fit,
                  se_log = pred_worker_vacancy$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_vacancy_df <- pred_worker_vacancy_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#plot
ggplot(pred_worker_vacancy_df, aes(x = prop_inactive_scaled, y = fit)) +
  geom_line(linewidth = 3) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  labs(
       x = "Proportion of vacant burrows (scaled)",
       y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Number of workers vs season

```
#create new data grid
worker_data_season<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  prop_inactive_scaled = mean(field_data$prop_inactive_scaled, na.rm = TRUE),
  bee_season_type = unique(field_data$bee_season_type)
)

# Get predicted probabilities and standard errors
pred_worker_season <- predict(workers_noArea,
                              newdata = worker_data_season,
                              type = "link",
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_worker_season_df <- cbind(worker_data_season,
                 fit_log = pred_worker_season$fit,
                 se_log = pred_worker_season$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_season_df <- pred_worker_season_df %>%
  mutate(
```

```
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

ggplot(pred_worker_season_df, aes(x = bee_season_type, y = fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper),
                width = 0.2, position = position_dodge(0.6), linewidth = 1.5) +
  labs(
       x = "Bee season",
       y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 23)
```



Findings

Number of nest-searching queens, number of rodent burrows and the proportion of burrow vacancy does not significantly predict number of workers. Significantly more workers during worker season than during nest-searching season. Significant positive effect of bombus-relevant floral abundance on worker abundance (p=0.0012).

Final Path Diagram